

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: LẬP TRÌNH VỚI PYTHON
MÃ HỌC PHẦN: INT13162**

**ĐỀ TÀI:
XÂY DỰNG CÔNG CỤ QUÉT CỔNG PORTS SCANNER**

Các sinh viên thực hiện:

B22DCAT127 – Nguyễn Xuân Hoàng

B22DCAT147 – Nguyễn Trường Huy

B22DCAT159 – Nguyễn Thanh Kiên

Tên lớp: D22-037

Tên nhóm: 03

Tên nhóm bài tập: 3

Giảng viên hướng dẫn: Thạc sĩ Vũ Minh Mạnh

HÀ NỘI 4-2025

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN BTL

TT	Công việc / Nhiệm vụ	SV thực hiện	Thời hạn hoàn thành
1	- Tổng hợp báo cáo - Xây dựng chức năng Network Scan, Detect Firewall	Nguyễn Xuân Hoàng	16/4/2025
2	- Xây dựng chức năng Single IP Scan, Shodan Lookup - Hỗ trợ tổng hợp báo cáo	Nguyễn Trường Huy	16/4/2025
3	- Xây dựng giao diện, tích hợp các chức năng, sửa lỗi - Làm slide thuyết trình	Nguyễn Thanh Kiên	16/4/2025

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	SV thực hiện	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Nguyễn Xuân Hoàng	5	4	3	4	3
2	Nguyễn Trường Huy	4	4	4	4	3
3	Nguyễn Thanh Kiên	5	4	4	4	3

Ghi chú:

- Thái độ tham gia: Đánh giá điểm thái độ tham gia công việc chung của nhóm (từ 0: không tham gia, đến 5: chủ động, tích cực).
- Mức hoàn thành CV: Đánh giá điểm mức độ hoàn thành công việc được giao (từ 0: không hoàn thành, đến 5: hoàn thành xuất sắc).
- Kỹ năng giao tiếp: Đánh giá điểm khả năng tương tác, giao tiếp trong nhóm (từ 0: không hoặc giao tiếp rất yếu, đến 5: giao tiếp xuất sắc).
- Kỹ năng hợp tác: Đánh giá điểm khả năng hợp tác, hỗ trợ lẫn nhau, giải quyết mâu thuẫn, xung đột
- Kỹ năng lãnh đạo: Đánh giá điểm khả năng lãnh đạo (từ 0: không có khả năng lãnh đạo, đến 5: có khả năng lãnh đạo tốt, tổ chức và điều phối công việc trong nhóm hiệu quả).

MỤC LỤC

MỤC LỤC.....	3
DANH MỤC CÁC HÌNH VẼ.....	4
DANH MỤC CÁC BẢNG BIỂU	5
DANH MỤC CÁC TỪ VIẾT TẮT.....	6
MỞ ĐẦU.....	8
CHƯƠNG 1. TỔNG QUAN VỀ CỤ QUÉT CỔNG PORTS SCANNER.....	9
1.1 Giới thiệu.....	9
1.2 Kiến trúc và những chức năng chính của PSC	9
1.3 Kết chương	10
CHƯƠNG 2. CÁC CHỨC NĂNG CHÍNH CỦA PSC	11
2.1 Khái quát	11
2.2 Single IP Scan	11
2.2.1 Mô tả	11
2.2.1.1 TCP Connect Scan.....	11
2.2.1.2 SYN (Stealth) Scan	11
2.2.1.3 XMAS Scan, FIN Scan, NULL Scan.....	12
2.2.1.4 UDP Scan	13
2.2.2 Cách thức hoạt động.....	14
2.2.2.1 SYN (Stealth) Scan	14
2.2.2.2 TCP Connect Scan	15
2.2.2.3 XMAS Scan, FIN Scan, NULL Scan.....	15
2.2.2.4 UDP Scan	17
2.3 Network Scan	17
2.3.1 Mô tả	17
2.3.1.1 Hàm phát hiện dịch vụ	18
2.3.1.2 Hàm phát hiện hệ điều hành.....	18
2.3.1.3 Hàm tìm thiết bị.....	19
2.3.1.4 Hàm vẽ sơ đồ mạng.....	19
2.3.2 Cách thức hoạt động.....	20
2.3.2.1 Hàm phát hiện dịch vụ	20
2.3.2.2 Hàm phát hiện hệ điều hành.....	21
2.3.2.3 Hàm tìm thiết bị.....	22
2.3.2.4 Hàm vẽ sơ đồ mạng.....	24

2.4 Shodan Lookup	25
2.4.1 Mô tả	25
2.4.2 Cách thức hoạt động.....	25
2.5 Detect Firewall	27
2.5.1 Mô tả	27
2.5.2 Cách thức hoạt động.....	27
2.6 Kết chương	28
CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ	29
3.1 Triển khai thử nghiệm công cụ	29
3.1.1 Thử nghiệm với chức năng Single IP Scan	29
3.1.2 Thử nghiệm với chức năng Network Scan.....	40
3.1.3 Thử nghiệm với chức năng Shodan Lookup	43
3.1.4 Thử nghiệm với chức năng Detect Firewall.....	44
3.2 Kết chương	46
KẾT LUẬN	47
TÀI LIỆU THAM KHẢO	48

DANH MỤC CÁC HÌNH VẼ

Hình 1: Công cụ Ports Scanner	9
Hình 2: Phản hồi thăm dò – Trạng thái chỉ định với SYN Scan	12
Hình 3: Phản hồi thăm dò – Trạng thái chỉ định với NULL, FIN, XMAX Scan.....	13
Hình 4: Phản hồi thăm dò – Trạng thái chỉ định với UDP Scan	13
Hình 5: Mô hình bắt tay 3 bước (tạo lập kết nối).....	18
Hình 6: Cấu trúc gói tin IP	19
Hình 7 : Cấu trúc của gói tin ARP	19
Hình 8: Cấu trúc gói tin IP/ICMP	27
Hình 9: Mô tả phản hồi của gói ICMP	27
Hình 10: Giao diện Single IP Scan với 1 số cổng (TCP Connect Scan).....	29
Hình 11: Giao diện Single IP Scan với 1 dải cổng (TCP Connect Scan).....	30
Hình 12: Giao diện Single IP Scan với 1 số cổng (SYN Scan).....	31
Hình 13: Giao diện Single IP Scan với 1 dải cổng (SYN Scan)	32
Hình 14: Giao diện Single IP Scan với 1 số cổng (XMAS Scan)	33
Hình 15: Giao diện Single IP Scan với 1 dải cổng (XMAS Scan).....	34

Hình 16: Giao diện Single IP Scan với 1 số cổng (FIN Scan)	35
Hình 17: Giao diện Single IP Scan với 1 dải cổng (FIN Scan).....	36
Hình 18: Giao diện Single IP Scan với 1 số cổng (NULL Scan).....	37
Hình 19: Giao diện Single IP Scan với 1 dải cổng (NULL Scan).....	38
Hình 20: Giao diện Single IP Scan với 1 số cổng (UDP Scan).....	39
Hình 21: Giao diện Single IP Scan với 1 dải cổng.....	40
Hình 22: Giao diện Network Scan với dải địa chỉ trong mạng cục bộ.....	41
Hình 23: Đồ thị mạng với dải địa chỉ trong mạng cục bộ	41
Hình 24: Giao diện Network Scan với dải địa chỉ ngoài mạng cục bộ	42
Hình 25: Sơ đồ mạng với dải địa chỉ ngoài mạng cục bộ	42
Hình 26: Giao diện Shodan Lookup với địa chỉ IP 1.1.1.1	43
Hình 27: Giao diện Shodan Lookup với địa chỉ IP 8.8.8.8	44
Hình 28: Giao diện Detect Firewall với 1 số cổng.....	45
Hình 29: Giao diện Detect Firewall với 1 dải cổng	46

DANH MỤC CÁC BẢNG BIỂU

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
PSC	Ports Scanner	Công cụ quét cổng
TCP	Transmission Control Protocol	Giao thức điều khiển truyền tải
IP	Internet Protocol	Giao thức liên mạng
UDP	User Datagram Protocol	Một giao thức trong bộ giao thức TCP/IP
ICMP	Internet Control Message Protocol	Giao thức kiểm soát báo lỗi
ARP	Address Resolution Protocol	Giao thức phân giải địa chỉ IP thành MAC
SYN	Synchronize	Cờ trong TCP để bắt đầu kết nối
ACK	Acknowledgment	Cờ trong TCP dùng để xác nhận dữ liệu
RST	Reset	Cờ TCP để đóng kết nối đột ngột
XMAX	XMAS Scan	Một kiểu quét port dùng nhiều cờ TCP (FIN, URG, PSH)
FIN	Finish	Cờ TCP báo hiệu kết thúc kết nối
NULL	NULL Scan	Kiểu quét TCP không bật cờ nào
RFC	Request For Comments	Tài liệu mô tả tiêu chuẩn Internet
LAN	Local Area Network	Mạng cục bộ
OS	Operating System	Hệ điều hành
TTL	Time To Live	Thời gian sống của gói tin
MAC	Media Access Control	Địa chỉ phần cứng của thiết bị mạng
API	Application Programming Interface	Giao diện lập trình ứng dụng
IPS	Intrusion Prevention System	Hệ thống ngăn chặn xâm nhập
ASN	Autonomous System Number	Mã số định danh duy nhất được gán cho một hệ thống mạng tự trị
DNS	Domain Name System	Hệ thống phân giải tên miền
TOS	Type Of Service	Loại dịch vụ

LỜI CẢM ƠN

Thay mặt nhóm em xin gửi lời cảm ơn chân thành tới thầy Vũ Minh Mạnh – Giảng viên hướng dẫn bộ môn Lập trình với Python học kỳ 2 năm học 2024 – 2025. Cảm ơn thầy vì đã truyền đạt cho chúng em những kiến thức về bộ môn Lập trình với Python để giúp chúng em trau dồi kiến thức nhằm phục vụ những môn học trong tương lai.

MỞ ĐẦU

Công cụ quét cổng Pors Scanner (PSC) là một công cụ có nhiều chức năng giúp xác định cổng mở của thiết bị và những thông tin khác. PSC cung cấp những chức năng giúp cho người dùng kiểm tra các cổng mở của thiết bị mục tiêu, quét dải mạng tìm kiếm các thiết bị, tra cứu địa chỉ IP Public (địa chỉ IP công cộng) sẵn có trên Shodan, kiểm tra tường lửa có chặn cổng thiết bị hay không. Cụ thể, các chức năng chính của PSC bao gồm:

- Single IP Scan: sử dụng để dò tìm các cổng mở của thiết bị.
- Network Scan: xác định xem thiết bị nào đang chạy trên một dải mạng cụ thể và các thông tin khác.
- Shodan Lookup: tra cứu thông tin của địa chỉ IP công cộng.
- Detect Firewall: kiểm tra xem tường lửa thiết bị có chặn cổng hay không.

Báo cáo bài tập lớn gồm 3 chương với nội dung chính như sau:

- Chương 1 giới thiệu tổng quan về PSC, bao gồm các nội dung khái quát về kiến trúc, các chức năng chính của PSC.
- Chương 2 thực hiện trình bày chi tiết về những chức năng chính của PSC, trong đó bao gồm mô tả và cách thức hoạt động của từng chức năng cùng với các hàm thực hiện.
- Chương 3 thực hiện việc thử nghiệm công cụ với từng chức năng cụ thể của PSC.

CHƯƠNG 1. TỔNG QUAN VỀ CỤ QUÉT CỔNG PORTS SCANNER

1.1 Giới thiệu

PSC (Port Scanner) là một công cụ quét cổng được phát triển bằng ngôn ngữ lập trình Python, hoạt động dựa trên nguyên lý gửi các gói tin mạng đến máy đích và phân tích phản hồi nhận được để xác định trạng thái của các cổng. Bằng cách khai thác phản hồi đặc trưng của từng giao thức, PSC có thể phân biệt các cổng đang mở, đóng, bị lọc hoặc không rõ trạng thái. Công cụ hỗ trợ nhiều kỹ thuật quét phổ biến như TCP SYN Scan, TCP Connect Scan, UDP Scan, FIN Scan, NULL Scan và XMAS Scan, cho phép đánh giá toàn diện về bề mặt tấn công của hệ thống đích. Ngoài ra, PSC còn tích hợp khả năng thu thập thông tin dịch vụ, phát hiện tường lửa, và tích hợp thêm Shodan nhằm nâng cao hiệu quả trình sát mạng. PSC là một công cụ hữu ích trong cả nghiên cứu bảo mật lẫn kiểm thử hệ thống thực tế.



Hình 1: Công cụ Ports Scanner

1.2 Kiến trúc và những chức năng chính của PSC

PSC được cấu trúc theo từng chức năng, mỗi chức năng bao gồm nhiều hàm hoạt động kết hợp tạo thành các chức năng chính. PSC được thiết kế để có khả năng mở rộng và tùy chỉnh, thuận tiện cho việc nâng cấp. PSC được chia ra thành các chức năng chính bao gồm:

- Single IP Scan: sử dụng để dò tìm các cổng mở của thiết bị
- Network Scan: xác định xem thiết bị nào đang chạy trên một dải mạng cụ thể và các thông tin khác
- Shodan Lookup: tra cứu thông tin của địa chỉ IP công cộng
- Detect Firewall: kiểm tra xem tường lửa thiết bị có chặn cổng hay không

1.3 Kết chương

Phần này đã khái quát về PSC, kiến trúc và các tính năng chính của PSC. Đây là nền tảng quan trọng cho việc xây dựng và phát triển các chức năng của PSC.

CHƯƠNG 2. CÁC CHỨC NĂNG CHÍNH CỦA PSC

2.1 Khái quát

Như đã đề cập trong phần Mở đầu, chương này tập trung vào các nội dung thiết kế và cài đặt các chức năng chính của công cụ. Công cụ sẽ có 4 chức năng chính bao gồm:

- Single IP Scan: sử dụng để dò tìm các cổng mở của thiết bị
- Network Scan: xác định xem thiết bị nào đang chạy trên một dải mạng cụ thể và các thông tin khác
- Shodan Lookup: tra cứu thông tin của địa chỉ IP công cộng
- Detect Firewall: kiểm tra xem tường lửa thiết bị có chặn cổng hay không

2.2 Single IP Scan

2.2.1 Mô tả

Chức năng Single IP Scan (quét địa chỉ IP đơn) cho phép kiểm tra một hoặc nhiều cổng của thiết bị có địa chỉ IP nhập vào có mở hay không bằng cách sử dụng nhiều phương pháp quét bao gồm:

- TCP Connect Scan
- SYN (Stealth) Scan
- XMAS Scan
- NULL Scan
- FIN Scan
- UDP Scan

2.2.1.1 TCP Connect Scan

TCP Connect Scan là phương pháp quét cổng mạng dựa trên việc hoàn tất đầy đủ quá trình bắt tay ba bước (Three-way handshake) của giao thức TCP. Khi thực hiện, máy quét gửi gói tin SYN đến cổng mục tiêu; nếu nhận được phản hồi SYN-ACK, máy quét tiếp tục gửi ACK để hoàn thành việc thiết lập kết nối, rồi lập tức đóng kết nối này lại. Dựa trên phản hồi nhận được, TCP Connect Scan xác định trạng thái cổng như sau:

- Open (Mở): nhận SYN-ACK từ cổng mục tiêu.
- Closed (Đóng): nhận RST-ACK từ cổng mục tiêu.

TCP Connect Scan minh bạch, ổn định và tuân thủ quy trình kết nối mạng tiêu chuẩn, phù hợp cho các môi trường có chính sách bảo mật nghiêm ngặt hoặc yêu cầu về pháp lý rõ ràng.

2.2.1.2 SYN (Stealth) Scan

SYN Scan, còn gọi là "Half-open scan", là một kỹ thuật quét cổng phổ biến trong các công cụ ví dụ như Nmap. SYN Scan gửi một gói TCP với cờ SYN (yêu cầu kết nối) đến cổng mục tiêu, nhưng không hoàn tất quá trình bắt tay ba bước của TCP (Three-way handshake). Dựa vào phản hồi từ hệ thống đích, SYN Scan sẽ xác định trạng thái của cổng như sau:

Probe Response	Assigned State
TCP SYN/ACK response	open
TCP RST response	closed
No response received (even after retransmissions)	filtered
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	filtered

Hình 2: Phản hồi thăm dò – Trạng thái chỉ định với SYN Scan

- Open: Nhận gói tin phản hồi có cờ SYN/ACK
=> Cổng đang hoạt động và lắng nghe kết nối.
=> PSC sẽ gửi gói RST để kết thúc kết nối.
- Closed: Nhận gói tin phản hồi có cờ RST
=> Dịch vụ không hoạt động trên cổng này.
- Filtered: Không nhận được bất kỳ phản hồi nào (None) hoặc nhận được thông báo lỗi ICMP.
=> Cổng có thể bị chặn bởi tường lửa hoặc thiết bị lọc gói tin.

Trong giao thức ICMP, Type 3 là thông báo lỗi "Destination Unreachable" (đích không thể tiếp cận). Mỗi loại lỗi có thêm một mã phụ (ICMP code) giúp xác định cụ thể nguyên nhân không thể tiếp cận. PSC chỉ quan tâm tới các mã được in đậm:

0 Mạng không thể truy cập được

1* Máy chủ không thể truy cập được (Host unreachable)

2* Giao thức không thể truy cập được (Protocol unreachable)

3* Cổng không thể truy cập được (Port unreachable)

4 Cần phân mảnh nhưng không đặt bit phân mảnh

5 Tuyên nguồn không thành công

6 Mạng đích không xác định

7 Máy chủ đích không xác định

8 Máy chủ nguồn bị cô lập (lỗi thời)

9* Mạng đích bị cấm về mặt hành chính (Destination network administratively prohibited)

10* Máy chủ đích bị cấm về mặt hành chính (Destination host administratively prohibited)

11 Mạng không thể truy cập được đối với loại dịch vụ (TOS)

12 Máy chủ không thể truy cập được đối với TOS

13* Truyền thông bị cấm về mặt hành chính do lọc (Communication administratively prohibited by filtering)

14 Vi phạm thứ tự ưu tiên của máy chủ

15 Có hiệu lực cắt thứ tự ưu tiên

2.2.1.3 XMAS Scan, FIN Scan, NULL Scan

Các kỹ thuật quét cổng NULL, FIN và XMAS Scan khai thác một đặc điểm đặc biệt của chuẩn giao thức TCP (RFC 793) để phân biệt trạng thái cổng mở và đóng. Theo RFC 793, khi một cổng TCP đóng nhận được gói tin mà không có các bit SYN, RST hoặc ACK được đặt, cổng này sẽ trả về gói tin RST. Ngược lại, khi cổng mở nó sẽ không trả lời gì cả. PSC khai thác điều này bằng 3 loại quét:

- NULL Scan: không đặt bất kỳ cờ nào trong gói TCP.
- FIN Scan: chỉ đặt cờ FIN trong gói TCP.
- XMAS Scan: đặt các cờ FIN, PSH và URG, tạo thành gói tin đặc biệt giống cây thông Noel.

Ba loại quét này có hành vi giống hệt nhau ngoại trừ các cờ TCP được đặt trong các gói thăm dò.

Dựa vào phản hồi từ hệ thống đích, chúng sẽ xác định trạng thái của cổng như sau:

Probe Response	Assigned State
No response received (even after retransmissions)	open filtered
TCP RST packet	closed
ICMP unreachable error (type 3, code 1, 2, 3, 9, 10, or 13)	filtered

Hình 3: Phản hồi thăm dò – Trạng thái chỉ định với NULL, FIN, XMAX Scan

- Nhận TCP RST → cổng closed
- Nhận ICMP unreachable (type 3, code 1/2/3/9/10/13) → cổng filtered
- Không phản hồi (kể cả sau khi thử lại) → cổng open|filtered

Ưu điểm chính của các phương pháp này là khả năng vượt qua một số firewall và thiết bị lọc gói không kiểm tra trạng thái kết nối. Các firewall thường chặn những gói tin TCP có đặt bit SYN và không đặt ACK, nên những kỹ thuật này không bị chặn bởi các quy tắc phổ biến đó

Tuy nhiên, các kỹ thuật này cũng tồn tại hạn chế. Nhiều hệ thống, như Microsoft Windows, thiết bị Cisco, hay IBM OS/400, không tuân thủ hoàn toàn RFC 793, luôn trả về gói tin RST dù cổng mở hay đóng, khiến tất cả các cổng bị hiểu nhầm là đóng. Kỹ thuật này thường chỉ hiệu quả với các hệ thống Unix tuân thủ chặt chẽ RFC.

Một hạn chế nữa là không thể phân biệt rõ ràng giữa các cổng mở và các cổng bị lọc.

2.2.1.4 UDP Scan

UDP Scan là phương pháp quét cổng mạng dựa trên việc gửi một gói tin UDP và phân tích gói tin phản hồi. Khi thực hiện, PSC gửi các gói UDP đến các cổng mục tiêu. Dựa trên phản hồi, UDP sẽ xác định trạng thái của cổng như sau:

Probe Response	Assigned State
Any UDP response from target port (unusual)	open
No response received (even after retransmissions)	open filtered
ICMP port unreachable error (type 3, code 3)	closed
Other ICMP unreachable errors (type 3, code 1, 2, 9, 10, or 13)	filtered

Hình 4: Phản hồi thăm dò – Trạng thái chỉ định với UDP Scan

- Open: khi nhận được phản hồi UDP (hiếm).
- Closed: khi nhận được lỗi ICMP “port unreachable” (loại 3 mã 3).
- Filtered: khi nhận biết các ICMP lỗi khác (loại 3, mã 1, 2, 9, 10, 13).
- Open|Filtered: khi không có phản hồi nào, PSC không thể phân biệt giữa cổng đang mở (nhưng không phản hồi) và cổng được lọc.

Một nhược điểm lớn của phương pháp quét UDP là tốc độ xử lý chậm, do nhiều hệ điều hành, đặc biệt là Linux giới hạn tốc độ phản hồi ICMP nhằm tránh tiêu tốn tài nguyên hệ thống. Do đó quá trình quét có thể kéo dài đến vài phút để hoàn tất.

2.2.2 Cách thức hoạt động

2.2.2.1 SYN (Stealth) Scan

Code:

```
def syn_scan(target, port):
    sport = scapy.RandShort()
    resp = scapy.sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="S"), timeout=1, verbose=0)
    if resp is not None:
        if resp.haslayer(scapy.TCP):
            if resp[scapy.TCP].flags == 18:
                reset = scapy.sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="R"), timeout=1, verbose=0)
                return (port, "Open")
            elif resp[scapy.TCP].flags == 20:
                return (port, "Closed")
        elif resp.haslayer(scapy.ICMP):
            if int(resp.getlayer(scapy.ICMP).type) == 3 and int(resp.getlayer(scapy.ICMP).code) in [1, 2, 3, 9, 10, 13]:
                return (port, "Filtered")
    else:
        return (port, "Filtered")
```

Giải thích code:

- `scapy.RandShort()` sinh một số ngẫu nhiên từ 1–65535 làm port nguồn (sport) giúp tăng tính ẩn danh
- `resp = scapy.sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="S"), timeout=1, verbose=0)`
- `scapy.IP(dst=target)` tạo gói IP với đích là target
- `"/` là toán tử Scapy để xếp chồng các lớp giao thức
- `scapy.TCP(sport=sport, dport=port, flags="S")`: tạo gói TCP SYN với
- sport: cổng nguồn ngẫu nhiên.
- dport: cổng đích cần quét.
- `flags="S"`: đặt cờ SYN (yêu cầu bắt đầu kết nối TCP).
- `scapy.sr1(...)`: gửi gói và chờ một phản hồi duy nhất.
- `timeout=1`: chờ 1 giây.
- `verbose=0`: không in log.
- Kết quả phản hồi lưu trong `resp` (response)

Kiểm tra gói phản hồi:

- TH1: `if resp.haslayer(scapy.TCP)`: Có lớp TCP trong `resp`
- TH1.1: `if resp[scapy.TCP].flags == 18`:
 - + `flags == 18` → tương đương với SYN+ACK
 - + SYN = 2, ACK = 16 → 2 + 16 = 18 ⇒ Cổng mở
 - ⇒ Gửi RST để hủy kết nối ngay lập tức
- TH1.2: `resp[scapy.TCP].flags == 20`:
 - + `flags == 20` → tương đương RST + ACK (theo chuẩn RFC 793, cờ ACK luôn bật thể hiện đích đến đã nhận gói)
 - + RST = 4, ACK = 16 → 4 + 16 = 20
 - ⇒ Cổng đóng
- TH2: Lỗi ICMP loại 3 ⇒ Filtered

- TH3: resp là None: không nhận được phản hồi=> Filtered

2.2.2.2 TCP Connect Scan

Code:

```
def tcp_connect_scan(ip, port):  
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
    sock.settimeout(1)  
    result = sock.connect_ex((ip, port))  
    sock.close()  
    if result == 0:  
        return (port, "Open")  
    else:  
        return (port, "Closed")
```

Giải thích code:

- `sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)`.
Tạo một socket TCP (SOCK_STREAM) sử dụng IPv4 (AF_INET). Hệ điều hành bắt đầu chuẩn bị tài nguyên để thiết lập một kết nối TCP.
- `sock.settimeout(1)`.
Thiết lập thời gian chờ (timeout) cho hoạt động kết nối là 1 giây, tránh việc chương trình bị treo nếu đích không phản hồi. Nếu sau 1 giây không thiết lập được kết nối, `connect_ex()` sẽ trả về lỗi timeout.
- `result = sock.connect_ex((ip, port))`.
Thực hiện kết nối TCP đến địa chỉ IP và cổng chỉ định. Đây là nơi diễn ra bắt tay TCP 3 bước, được xử lý hoàn toàn bởi TCP stack của hệ điều hành, `connect_ex()` trả về mã lỗi:

- 0 → Kết nối thành công (cổng mở).
- 111, 10061, v.v. → Kết nối thất bại (cổng đóng hoặc bị từ chối).
- `sock.close()`: Đóng kết nối TCP ngay sau khi thiết lập (hoặc sau khi thất bại).
- `if result == 0: return (port, "Open") else: return (port, "Closed")`.

Dựa vào mã lỗi trả về từ `connect_ex()` để xác định trạng thái cổng

- + Nếu kết nối thành công (`result == 0`) → cổng Open.
- + Nếu thất bại (bị từ chối, timeout...) → cổng Closed.

2.2.2.3 XMAS Scan, FIN Scan, NULL Scan

Code:

```

def xmas_scan(target, port):
    sport = scapy.RandShort()
    resp = sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="FPU"), timeout=1, verbose=0)
    if resp is None:
        return (port, "Open|Filtered")
    elif resp.haslayer(scapy.TCP):
        if resp[scapy.TCP].flags == 20:
            return (port, "Closed")
    elif resp.haslayer(scapy.ICMP):
        if int(resp.getlayer(scapy.ICMP).type) == 3 and int(resp.getlayer(scapy.ICMP).code) in [1, 2, 3, 9, 10, 13]:
            return (port, "Filtered")

def fin_scan(target, port):
    sport = scapy.RandShort()
    resp = sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="F"), timeout=1, verbose=0)
    if resp is None:
        return (port, "Open|Filtered")
    elif resp.haslayer(scapy.TCP):
        if resp[scapy.TCP].flags == 20:
            return (port, "Closed")
    elif resp.haslayer(scapy.ICMP):
        if int(resp.getlayer(scapy.ICMP).type) == 3 and int(resp.getlayer(scapy.ICMP).code) in [1, 2, 3, 9, 10, 13]:
            return (port, "Filtered")

def null_scan(target, port):
    sport = scapy.RandShort()
    resp = sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags=""), timeout=1, verbose=0)
    if resp is None:
        return (port, "Open|Filtered")
    elif resp.haslayer(scapy.TCP):
        if resp[scapy.TCP].flags == 20:
            return (port, "Closed")
    elif resp.haslayer(scapy.ICMP):
        if int(resp.getlayer(scapy.ICMP).type) == 3 and int(resp.getlayer(scapy.ICMP).code) in [1, 2, 3, 9, 10, 13]:
            return (port, "Filtered")

```

Tất cả ba hàm đều có cấu trúc giống nhau chỉ khác mỗi cờ của gói tin được gửi.

Giải thích code:

- `scapy.RandShort()` sinh một số ngẫu nhiên từ 1–65535 làm port nguồn (sport) giúp tăng tính ẩn danh
- Gửi gói tin TCP và nhận phản hồi
 - + XMAS scan: cờ FIN, PSH, URG
`resp = scapy.sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="FPU"), timeout=1, verbose=0)`
 - + FIN scan: cờ FIN
`resp = scapy.sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags="F"), timeout=1, verbose=0)`
 - + NULL scan: không bật cờ nào
`resp = scapy.sr1(scapy.IP(dst=target) / scapy.TCP(sport=sport, dport=port, flags=""), timeout=1, verbose=0)`
- Kiểm tra gói phản hồi:
 - + TH1: `resp` là None: không nhận được phản hồi => Cổng Open|Filtered
 - + TH2: `elif resp.haslayer(scapy.TCP)`: Có lớp TCP trong `resp`

- if resp[scapy.TCP].flags == 20: flags == 20 → tương đương RST + ACK (theo chuẩn RFC 793, cờ ACK luôn bật thể hiện đích đến đã nhận gói).
 - RST = 4, ACK = 16 → 4 + 16 = 20 => Cổng đóng
- + TH3: Lỗi ICMP loại 3 => Filtered

2.2.2.4 UDP Scan

Code:

```
def udp_scan(target, port):
    sport = scapy.RandShort()
    resp = scapy.sr1(scapy.IP(dst=target) / scapy.UDP(sport=sport, dport=port), timeout=1, verbose=0)
    if resp is not None:
        if resp.haslayer(scapy.UDP):
            return (port, "Open")
        elif int(resp.getlayer(scapy.ICMP).type) == 3 and int(resp.getlayer(scapy.ICMP).code) == 3:
            return (port, "Closed")
        elif int(resp.getlayer(scapy.ICMP).type) == 3 and int(resp.getlayer(scapy.ICMP).code) in [1, 2, 9, 10, 13]:
            return (port, "Filtered")
    else:
        return (port, "Open|Filtered")
```

Giải thích code:

- scapy.RandShort() sinh một số ngẫu nhiên từ 1–65535 làm port nguồn (sport) giúp tăng tính ẩn danh
- Gửi gói tin UDP và nhận phản hồi
`resp = scapy.sr1(scapy.IP(dst=target) / scapy.UDP(sport=sport, dport=port), timeout=1, verbose=0)`
- Kiểm tra gói phản hồi:
 - + TH1: if resp.haslayer(scapy.UDP): Có lớp UDP trong resp => Cổng mở
 - + TH2: Lỗi ICMP loại 3 code 3 => Cổng đóng
 - + TH3: Lỗi ICMP loại 3 code 1, 2, 9, 10 hoặc 13 => Filtered
 - + TH4: resp là None: không nhận được phản hồi
=> Open|Filtered

2.3 Network Scan

2.3.1 Mô tả

Chức năng Network Scan sẽ quét một dải mạng từ địa chỉ IP bắt đầu và kết thúc và trả về một danh sách thiết bị với các thông số:

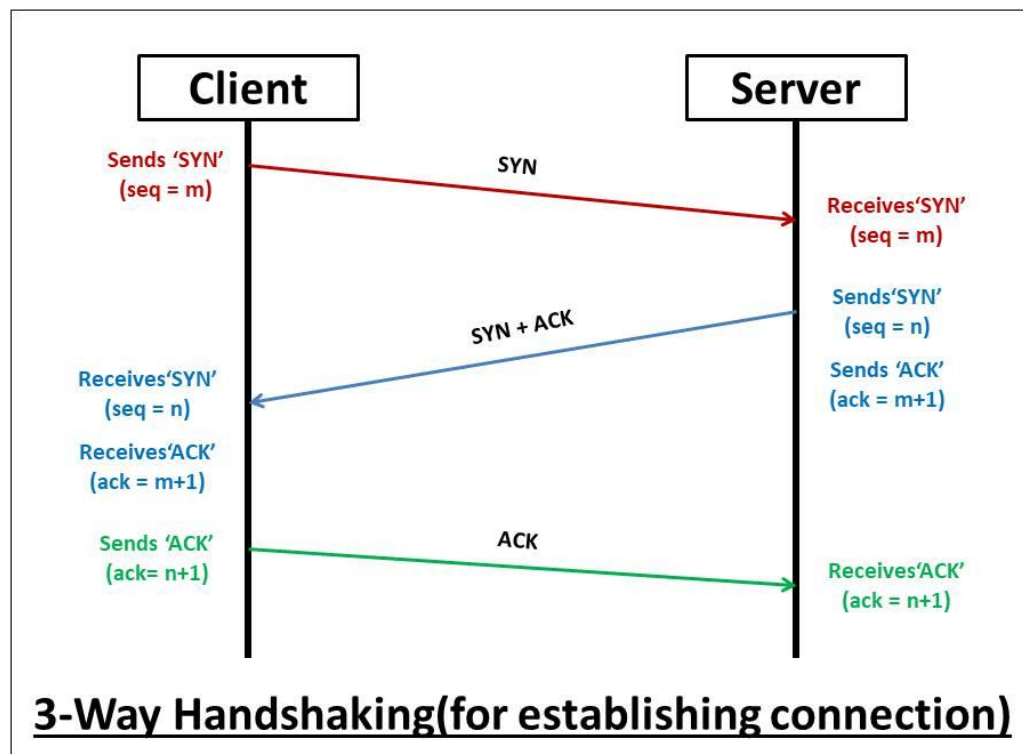
- IP: địa chỉ IP của thiết bị muốn kiểm tra.
- MAC: địa chỉ MAC của thiết bị muốn kiểm tra (nếu có).
- Hostname: tên thiết bị (nếu có).
- Status: trạng thái của thiết bị: Active – hoạt động, Inactive – không hoạt động.
- OS: hệ điều hành của thiết bị muốn kiểm tra (nếu có).
- Services: dịch vụ của thiết bị muốn kiểm tra cung cấp (nếu có).

Chức năng Network Scan được tạo thành từ các hàm chức năng riêng.

2.3.1.1 Hàm phát hiện dịch vụ

Hàm phát hiện dịch vụ với đầu vào là một địa chỉ IP của thiết bị muốn phát hiện dịch vụ. Trong hàm này chỉ thực hiện việc phát hiện những dịch vụ phổ biến (có thể bổ sung thêm).

Hàm thực hiện kết nối theo mô hình bắt tay ba bước bằng việc gửi gói tin TCP chứa cờ SYN yêu cầu kết nối và nếu nhận được gói tin phản hồi có cờ SYN-ACK thì gửi lại gói TCP có cờ ACK để xác nhận kết nối rồi gửi thêm gói tin TCP có chứa payload yêu cầu cổng thiết bị cung cấp thông tin dịch vụ đang cung cấp (nếu có).

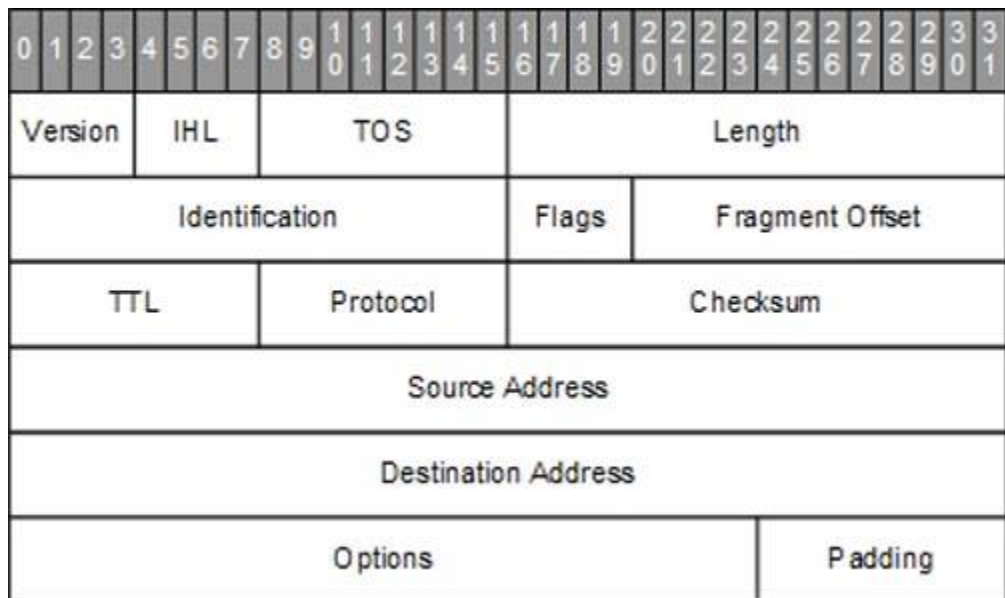


Hình 5: Mô hình bắt tay 3 bước (tạo lập kết nối)

2.3.1.2 Hàm phát hiện hệ điều hành

Hàm phát hiện hệ điều hành với đầu vào là một địa chỉ IP của thiết bị muốn phát hiện hệ điều hành.

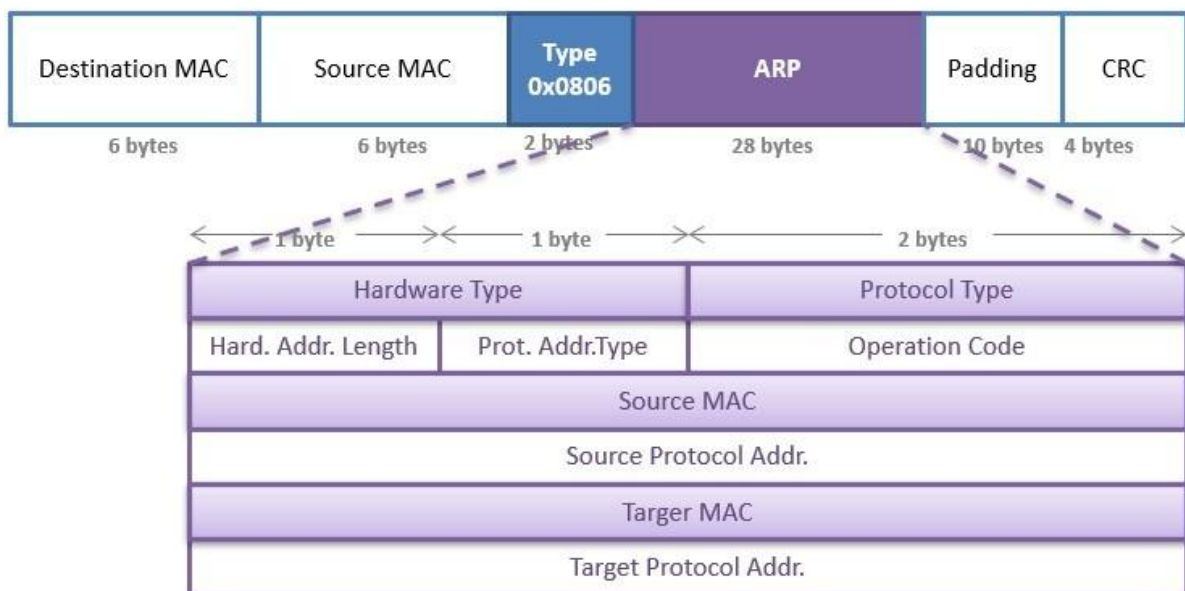
Trong hàm phát hiện hệ điều hành sẽ tiến hành gửi gói tin TCP tới danh sách các cổng của thiết bị (có thể mở rộng) nhằm phân tích thông số Time To Live (TTL) của gói tin IP phản hồi để xác định hệ điều hành của thiết bị.



Hình 6: Cấu trúc gói tin IP

2.3.1.3 Hàm tìm thiết bị

Hàm tìm thiết bị với đầu vào là một dải địa chỉ IP có địa chỉ IP bắt đầu và địa chỉ IP kết thúc cùng với địa chỉ dải mạng cục bộ (LAN). Trong hàm tìm thiết bị sẽ sử dụng gói tin ARP (để lấy địa chỉ MAC), hàm phát hiện dịch vụ và hàm phát hiện hệ điều hành để xác định thông số của các thiết bị mang địa chỉ IP: MAC (nếu có) , Hostname (nếu có), Status, OS, Services.



Hình 7 : Cấu trúc của gói tin ARP

2.3.1.4 Hàm vẽ sơ đồ mạng

Hàm vẽ sơ đồ mạng với đầu vào là danh sách các thiết bị với các thông số của mỗi thiết bị bao gồm:

- IP: địa chỉ IP của thiết bị muốn kiểm tra.
- MAC: địa chỉ MAC của thiết bị muốn kiểm tra (nếu có).
- Hostname: tên thiết bị (nếu có).
- Status: trạng thái của thiết bị :Active – hoạt động, Inactive – không hoạt động.
- OS: hệ điều hành của thiết bị muốn kiểm tra (nếu có).
- Services: dịch vụ của thiết bị muốn kiểm tra cung cấp (nếu có).

Hàm sử dụng việc kết nối những thiết bị có trạng thái Active (hoạt động) để mô phỏng sơ đồ mạng dưới dạng đồ thị vô hướng với mỗi thiết bị là một nút.

2.3.2 Cách thức hoạt động

2.3.2.1 Hàm phát hiện dịch vụ

Code:

```
def detect_services(ip):
    services = {}
    detected = {}
    service_payloads = {
        21: "HELP\r\n",                # FTP
        22: "\r\n",                    # SSH
        23: "\r\n",                    # Telnet
        25: "EHLO test\r\n",           # SMTP
        80: "GET / HTTP/1.1\r\nHost: {}".format(ip).encode(), # HTTP
        110: "USER test\r\n",          # POP3
        143: ". LOGIN test test\r\n",  # IMAP
        443: "\r\n",                   # HTTPS
        3306: "\x00",                  # MySQL
    }

    for port, payload in service_payloads.items():
        syn_packet = IP(dst=ip) / TCP(dport=port, flags="S")
        response = sr1(*args: syn_packet, timeout=1, verbose=0)
        if response and response.haslayer(TCP) and response[TCP].flags == 0x12:
            ack = IP(dst=ip) / TCP(dport=port, flags="A", seq=100, ack=response.seq + 1)
            send(ack, verbose=0)
            psh = IP(dst=ip) / TCP(dport=port, sport=12345, flags="PA", seq=101, ack=response.seq + 1) / Raw(load=payload)
            banner_response = sr1(*args: psh, timeout=2, verbose=0)
            if banner_response and banner_response.haslayer(Raw):
                try:
                    banner = banner_response[Raw].load.decode(errors="ignore").strip()
                    services[port] = f"Open ({banner})"
                except:
                    services[port] = f"Open (Unreadable banner)"
            else:
                services[port] = "Open (No banner)"
        elif response is None:
            services[port] = "No response"
        else:
            services[port] = "Closed or Filtered"
    for port, status in services.items():
        if "Open" in status:
            detected[port] = status

    return detected
```

Giải thích đoạn code:

Đầu tiên khởi tạo một từ điển dịch vụ rỗng, phát hiện rỗng, một từ điển chứa những cổng dịch vụ phổ biến và các payload được thiết kế riêng cho dịch vụ mạng đó để yêu cầu cung cấp thông tin về dịch vụ. Với mỗi cổng trong từ điển dịch vụ phổ biến sử dụng gửi gói tin TCP với cờ SYN nhằm yêu cầu kết nối qua quá trình bắt tay ba bước:

- Nếu có gói tin phản hồi và gói tin có chứa TCP với cờ “0x12” tương ứng với cờ SYN-ACK thì tức là cổng dịch vụ mở và tiến hành gửi gói tin TCP chứa cờ ACK với thông số seq = seq của gói tin phản hồi + 1 để xác nhận kết nối sau đó tiến hành gửi gói tin TCP với cờ PA với thông số seq = seq của gói tin phản hồi + 1 cùng với payload riêng của cổng để yêu cầu thông tin dịch vụ mà cổng của thiết bị cung cấp:
 - + Nếu có gói tin phản hồi và có chứa raw (dữ liệu) bên trong thì thông tin dịch vụ là giữ liệu phản hồi của gói tin mã cổng dịch vụ của thiết bị đã cung cấp và tiến hành gán cổng mở với dịch vụ đó.
 - + Nếu gói tin giải mã bị lỗi không thể giải mã thì tiến hành gán cổng mở với thông tin dịch vụ không xác định.
 - + Nếu không có gói tin chứa dữ liệu thì tiến hành gán cổng mở với thông tin dịch vụ không có.
- Nếu nhận được gói tin phản hồi trả về “None” thì gán cổng với thông tin không phản hồi.
- Với những trường hợp khác thì gán cổng với thông tin đã đóng hoặc bị chặn.

Cuối cùng trả về các cổng mở.

2.3.2.2 Hàm phát hiện hệ điều hành

Code:

```
def detect_os(ip):
    common_ports = [ 21, 22, 25, 80, 110, 143, 443, 445, 3306, 3389, 5432 ]
    for port in common_ports:
        packet = IP(dst=ip) / TCP(dport=port, flags="S")
        response = sr1(*args: packet, timeout=1, verbose=0)
        if response:
            ttl = response.ttl
            if ttl <= 64:
                return (f" Linux/Unix-based OS (TTL={ttl})")
            elif ttl <= 128:
                return (f" Windows OS (TTL={ttl})")
            else:
                return (f" Network Device / Router (TTL={ttl})")
    return (f" Undetermined")
```

Giải thích đoạn code:

Đầu tiên ta sử dụng gói tin TCP với cờ SYN tới các cổng phổ biến nhằm yêu cầu kết nối theo mô hình bắt tay 3 bước nhưng ở đây chỉ cần gửi gói tin để nhận gói tin TCP phản hồi từ thiết bị muốn xác định:

- Nếu có gói tin phản hồi thì đó chắc chắn là gói tin IP, tiến hành lấy thông số TTL (Time To Live) của gói IP để phân tích.
 - + Nếu thông số TTL <= 64 thì hệ điều hành là Linux/Unix-based OS (thường có TTL mặc định là 64).
 - + Nếu thông số TTL <= 128 thì hệ điều hành thường là Windows OS (thường có TTL mặc định là 128).

- + Nếu thông số TTL lớn hơn thì có thể là Router / Network Device (thường có TTL lớn hơn).
- Nếu không có phản hồi thì trả về thông số thiết bị đó không xác định.

2.3.2.3 Hàm tìm thiết bị

Code:

```
def find_devices(start_ip, end_ip, local_network):
    try:
        start = ipaddress.IPv4Address(start_ip)
        end = ipaddress.IPv4Address(end_ip)
    except ipaddress.AddressValueError:
        print("IP address Invalid")
        return []

    if start > end:
        print("IP address Invalid.")
        return []

    network = ipaddress.ip_network(local_network, strict=0)
    if start in network and end in network:
        devices = []

        ip_list = [str(ip) for ip in range(int(start), int(end) + 1)]
        arp_request = ARP(pdst=ip_list)
        broadcast = Ether(dst="ff:ff:ff:ff:ff:ff")
        packet = broadcast / arp_request
        answered, unanswered = srp(packet, timeout=2, verbose=0)

        for sent, received in answered:
            try:
                hostname = socket.gethostbyaddr(received.psrc)[0]
                status = "Active"
            except socket.herror:
                hostname = "Unknown"
                status = "Active"

            os_guess = detect_os(received.psrc)
            services = detect_services(received.psrc)

            devices.append({
                "IP": received.psrc,
                "MAC": received.hwsrc,
                "Hostname": hostname,
                "Status": status,
                "OS": os_guess,
                "Service": services
            })
```

```

        for sent in unanswered:
            ip=str(ipaddress.IPv4Address(int(sent[0].pdst)))
            devices.append({
                "IP": ip,
                "MAC": "N/A",
                "Hostname": "Unknown",
                "Status": "Inactive",
                "OS": "Unknown",
                "Service": {},
            })
    else:
        devices = []

    ip_list = [str(ip) for ip in range(int(start), int(end) + 1)]
    for ip in ip_list:
        ip = str(ipaddress.IPv4Address(int(ip)))
        try:
            hostname = socket.gethostbyaddr(ip)[0]
        except socket.herror:
            hostname = "Unknown"
        services = detect_services(ip)
        if services:
            os_guess = detect_os(ip)
            devices.append({
                "IP": ip,
                "MAC": "N/A",
                "Hostname": hostname,
                "Status": "Active",
                "OS": os_guess,
                "Service": services,
            })
        else:
            devices.append({
                "IP": ip,
                "MAC": "N/A",
                "Hostname": "Unknown",
                "Status": "Inactive",
                "OS": "Unknown",
                "Service": {},
            })

```

```

devices.sort(key=lambda x: x["Status"]=="Inactive")
if not devices:
    print("No devices found.")
else:
    print(f'{ "IP Address":<15} { "MAC Address":<20} { "Hostname":<20} { "Status":<20} { "OS":<40} { "Service":<50}'}
    print("-" * 180)
    for device in devices:
        services_str = '{ ' + ', '.join([f'{port}: {name}' for port, name in device['Service'].items()]) + '}' if device['Service'] else '{}'
        print(f'{device["IP"]:<15} {device["MAC"]:<20} {device["Hostname"]:<20} {device["Status"]:<20} {device["OS"]:<40} {services_str.center(50)}')

    return devices

```

Giải thích đoạn code:

Đầu tiên là phát hiện thiết bị hoạt động:

- Với dải địa chỉ nằm trong mạng cục bộ thì sử dụng việc gửi gói tin ARP mang địa chỉ IP của các thiết bị cần tìm rồi gửi tới địa chỉ broadcast (gửi cho tất cả các thiết bị trong mạng) và chờ phản hồi: Vì các gói ARP được gửi đi và nhận lại dưới dạng một danh sách với những gói tin phản hồi và không phản hồi nên ta tiến hành xử lý từng phần:
 - + Với những gói tin phản hồi lại sử dụng phân giải ngược tên miền để lấy hostname (nếu có) và đặt trạng thái của thiết bị với IP nguồn của phản hồi là hoạt động (active) sau đó tiến hành gọi các hàm phát hiện hệ điều hành, phát hiện dịch vụ

với địa chỉ IP nguồn của phản hồi và gán lại các thông số cho thiết bị mang IP đó (IP, địa chỉ MAC, hostname, trạng thái, hệ điều hành, dịch vụ).

- + Với những gói tin không phản hồi tiến hành gán những thông số cho thiết bị mang IP đó với các thông số không tích cực (không hoạt động).
- Với dải địa chỉ nằm ngoài mạng cục bộ thì sử dụng hàm phát hiện dịch vụ:
 - + Nếu phát hiện thiết bị mang IP cần tìm kiểm có bất kì dịch vụ nào thì thiết bị ấy đang hoạt động (active), thực hiện truy vấn ngược để tìm hostname (nếu có) sau đó thực hiện hàm phát hiện hệ điều hành để tìm hệ điều hành thiết bị. Vì không thể lấy địa chỉ MAC nên đặt địa chỉ MAC là “N/A” và dịch vụ tìm được sẽ là dịch vụ của thiết bị.
 - + Nếu thiết bị mang IP không có bất kì dịch vụ nào thì gán lại các thông số cho thiết bị mang IP đó (IP, địa chỉ MAC, hostname, trạng thái, hệ điều hành, dịch vụ).

Cuối cùng trả về một danh sách các thiết bị mang địa chỉ IP với các thông số của từng thiết bị (IP, địa chỉ MAC, hostname, trạng thái, hệ điều hành, dịch vụ).

2.3.2.4 Hàm vẽ sơ đồ mạng

Code:

```
def draw_network_map(devices):
    active_devices = [device for device in devices if device['Status'] == "Active"]
    if not active_devices:
        print("No active devices")
        return

    G = nx.Graph()
    for device in active_devices:
        label = f"{device['IP']}\n{device['Hostname']}" if device['Hostname'] != "Unknown" else device['IP']
        G.add_node(label)

    gateway = None
    for device in active_devices:
        if device['IP'].endswith(".1") or device['IP'].endswith(".254"):
            gateway = f"{device['IP']}\n{device['Hostname']}" if device['Hostname'] != "Unknown" else device['IP']
            break
    if not gateway:
        gateway = "Gateway"
        G.add_node(gateway)

    for device in active_devices:
        node_label = f"{device['IP']}\n{device['Hostname']}" if device['Hostname'] != "Unknown" else device['IP']
        if node_label != gateway:
            G.add_edge(gateway, node_label)

    pos = nx.spring_layout(G, seed=42)
    fig = plt.figure(figsize=(10, 7))
    fig.canvas.manager.set_window_title("Network Map <3")
    nx.draw(G, pos, with_labels=True, node_color="skyblue", node_size=1500, font_size=10, font_weight="bold")
    plt.title("Network Map")
    plt.show()
```

Giải thích đoạn code:

Đầu tiên lấy những thiết bị hoạt động trong danh sách dùng thư viện NetworkX để tạo đồ thị vô hướng “G” sau đó thêm các nút đại diện cho các thiết bị vào đồ thị.

- Mỗi nút là một thiết bị hoạt động có nhãn là địa chỉ IP và hostname (nếu có).
 - + Nếu trong các thiết bị có gateway (thường có IP kết thúc bằng “.1” hoặc “.254”) sẽ là nút gateway.
 - + Nếu không có thì tạo một nút gateway và thêm các cạnh nối từ gateway tới các thiết bị đang hoạt động.

Cuối cùng trả về một đồ thị vô hướng gồm các thiết bị hoạt động được kết nối với nhau tại nút gateway.

2.4 Shodan Lookup

2.4.1 Mô tả

Mặc dù các phương thức quét công đã nói trên có thể quét public IP, nhưng có thể gây ra nhiều rủi ro nghiêm trọng về pháp lý. Dù chỉ nhằm vào mục đích học tập hoặc nghiên cứu, hành động gửi gói tin đến hệ thống không thuộc quyền sở hữu của người quét có thể bị coi là xâm nhập trái phép, đặc biệt nếu tạo ra lưu lượng truy cập lớn, làm ảnh hưởng đến dịch vụ hoặc bị hệ thống giám sát ghi nhận. Một số quốc gia và tổ chức áp dụng quy định nghiêm ngặt đối với hành vi quét công mà không được phép, và hoạt động này có thể dẫn đến cảnh báo, khóa IP hoặc truy tìm trách nhiệm pháp lý.

Để tránh những rủi ro xảy ra, nhóm đã quyết định tích hợp Shodan, một công cụ tìm kiếm chuyên dụng cho các thiết bị và dịch vụ khai báo trên Internet vào hệ thống quét công của mình. Shodan thu thập dữ liệu từ các cổng dịch vụ trên toàn thế giới và lưu trữ sẵn có trong cơ sở dữ liệu, cho phép người dùng truy cập thông tin mà không cần quét trực tiếp.

Bằng cách sử dụng Shodan API, ta có thể nghiên cứu thông tin về IP chủ sở hữu của tổ chức, dịch vụ đang mở, hệ điều hành, tên miền, vị trí địa lý và cổng hoạt động theo cách an toàn và hợp pháp, đảm bảo tuân thủ các nguyên tắc đạo đức và quy định pháp lý trong an toàn thông tin

2.4.2 Cách thức hoạt động

Hàm `shodan_lookup` được xây dựng nhằm mở rộng khả năng của công cụ quét công, cho phép tra cứu thông tin chi tiết về một địa chỉ IP công khai thông qua Shodan API.

Code:

```

SHODAN_API_KEY = "NSMUvugy9X98QB17zsq2k2MtNjkdGHHK"
api = shodan.Shodan(SHODAN_API_KEY)

def shodan_lookup(ip):
    try:
        result = api.host(ip)
        output = []
        output.append(f"Thông tin về địa chỉ IP: {ip}")
        output.append("-" * 60)
        output.append(f"Tổ chức: {result.get('org', 'Không có thông tin')}")
        output.append(f"Nhà cung cấp dịch vụ (ISP): {result.get('isp', 'Không có thông tin')}")
        output.append(f"Hệ điều hành: {result.get('os', 'Không có thông tin')}")
        output.append(f"Vị trí: {result.get('city', 'Không có thông tin')}, {result.get('country_name', 'Không có thông tin')}")
        output.append(f"ASN: {result.get('asn', 'Không có thông tin')}")
        output.append(f"Tên miền liên quan: " + ", ".join(result.get("domains", ["Không có thông tin"])))
        output.append(f"Danh sách hostname: " + ", ".join(result.get("hostnames", ["Không có thông tin"])))
        output.append(f"Số lượng cổng mở: {len(result.get('ports', []))}")
        output.append(f"Danh sách cổng mở:")
        output.append("-" * 30)

        for service in result.get("data", []):
            port = service.get("port")
            protocol = service.get("_shodan", {}).get("module", "Không xác định")
            output.append(f"Cổng: {port} - Giao thức: {protocol}")

        output.append("-" * 60)
        return "\n".join(output)

    except shodan.APIError as error:
        return f"Lỗi Shodan API: {error}"

```

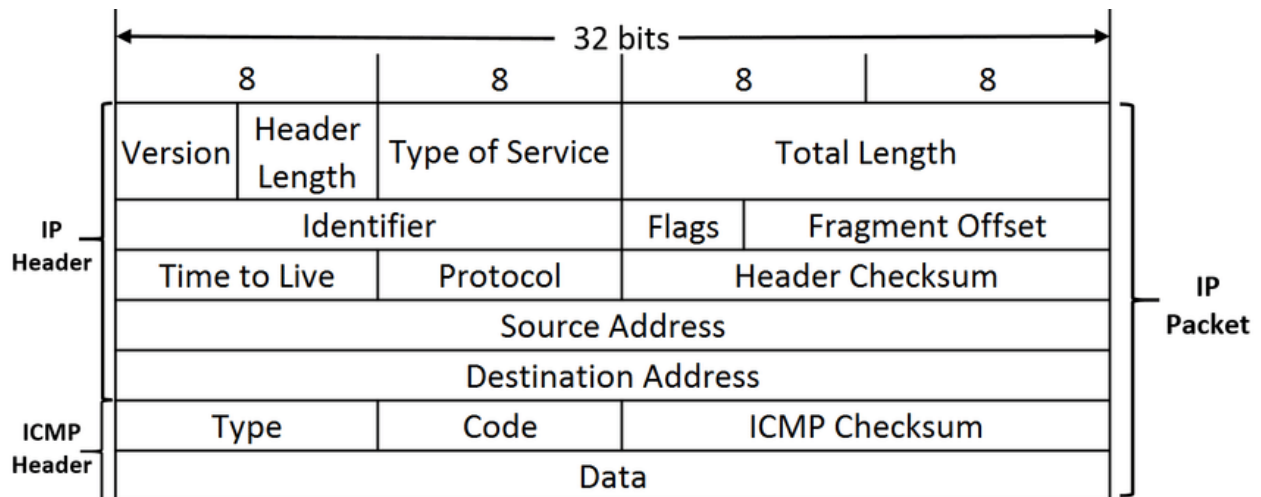
- Khởi tạo kết nối với Shodan, sử dụng API key cá nhân đã đăng ký trên trang Shodan.io:
 - `api = shodan.Shodan(SHODAN_API_KEY)`
- Gọi `api.host(ip)` để lấy toàn bộ thông tin liên quan đến địa chỉ IP đầu vào và lưu vào `result`:
 - `result = api.host(ip)`
- Trích xuất dữ liệu:
 - + Tổ chức sở hữu (org): Cho biết tổ chức quản lý IP, ví dụ như Google, Microsoft, hay một ISP cụ thể.
 - + Nhà cung cấp dịch vụ (ISP): Nhà mạng cung cấp đường truyền Internet cho IP đó.
 - + Hệ điều hành (OS): Hệ điều hành được Shodan suy đoán từ banner dịch vụ, nếu có thể xác định.
 - + Vị trí địa lý: Bao gồm thành phố (city) và quốc gia (country_name), hỗ trợ định vị vị trí vật lý của thiết bị.
 - + ASN (Autonomous System Number): Mã số định danh của hệ thống mạng mà IP này thuộc về
 - + Tên miền liên quan (domains): các tên miền đã được ánh xạ tới IP.
 - + Danh sách hostname (hostnames): Tên định danh máy chủ trong hệ thống DNS
 - + Số lượng cổng mở (ports): Đếm tổng số cổng mà Shodan phát hiện đang mở.
 - + Danh sách chi tiết các cổng: Với mỗi dịch vụ đang hoạt động, hàm sẽ in:
 - Số hiệu cổng (port): Ví dụ: 80, 443, 22...
 - Giao thức hoặc mô-đun quét (shodan.module): Ví dụ: http, ssh, ftp, telnet,...

2.5 Detect Firewall

2.5.1 Mô tả

Chức năng Detect Firewall sẽ kiểm tra một cổng của một thiết bị có địa chỉ IP cần kiểm tra có tường lửa chặn truy cập cổng hay không bằng việc gửi gói tin TCP rồi phân tích phản hồi để nhận được kết quả.

Có thể căn cứ vào phản hồi của gói tin ICMP để phát hiện cổng của thiết bị có tường lửa chặn hay không.



Hình 8: Cấu trúc gói tin IP/ICMP

ICMP Type	ICMP Code	Description
0	0	Echo Reply (used by ping)
3	0	Destination Network Unreachable
3	1	Destination Host Unreachable
3	3	Destination Port Unreachable
8	0	Echo Request (used by ping)
11	0	TTL Expired (used by traceroute)

ICMP Types and Codes

Hình 9: Mô tả phản hồi của gói ICMP

2.5.2 Cách thức hoạt động

Hàm phát hiện tường lửa:

```
def detect_firewall(ip, port, cancel_event=None):
    if cancel_event and cancel_event.is_set():
        return f"Port {port}: Scan cancelled by user."
    packet = IP(dst=ip) / TCP(dport=port, flags="S")
    response = sr1(*args, packet, timeout=1, verbose=0)
    if cancel_event and cancel_event.is_set():
        return f"Port {port}: Scan cancelled by user."
    if response is None:
        return f"Port {port}: Firewall detected (No Response)"
    if response.haslayer(TCP):
        if response.getlayer(TCP).flags == 0x14:
            return f"Port {port}: Firewall detected/Closed Port"
        elif response.getlayer(TCP).flags == 0x12:
            return f"Port {port}: No Firewall (Open)"
    if response.haslayer(ICMP):
        icmp_layer = response.getlayer(ICMP)
        if icmp_layer.type == 3 and icmp_layer.code == 3:
            return f"Port {port}: Port Closed (ICMP Port Unreachable)"
        else:
            return f"Firewall Detected (ICMP Error) at port:{port}"
    return f"Port {port}: Undetermined"
```

Đầu tiên ta sử dụng gói tin TCP với cờ SYN tới cổng muốn kiểm tra nhằm yêu cầu kết nối theo mô hình bắt tay 3 bước nhưng ở đây chỉ cần gửi gói tin để nhận các gói tin phản hồi từ thiết bị:

- Nếu nhận được gói tin phản hồi là "None" -> tường lửa của hệ thống đã chặn truy cập cổng này (phát hiện tường lửa).
 - + Nếu phản hồi là có chứa gói TCP:
 - Nếu gói TCP có chứa cờ "0x14" là cờ ACK + RST thì tức là hệ thống có tường lửa và đã không cho phép kết nối hoặc cổng đóng bằng cách trả về gói tin chứa cờ ACK + RST.
 - Nếu gói TCP có chứa cờ "0x12" là cờ SYN + ACK thì tức là hệ thống không chặn truy cập vào cổng.
- Nếu phản hồi có chứa gói ICMP:
 - + Nếu ICMP có type=3 và code=3 thì tức cổng của hệ thống đóng.
 - + Nếu ICMP có type và code khác thì tức tường lửa của hệ thống đã chặn truy cập cổng bằng cách gửi trả gói tin chứa ICMP.
- Với các trường hợp khác thì trả về cổng không xác định.

2.6 Kết chương

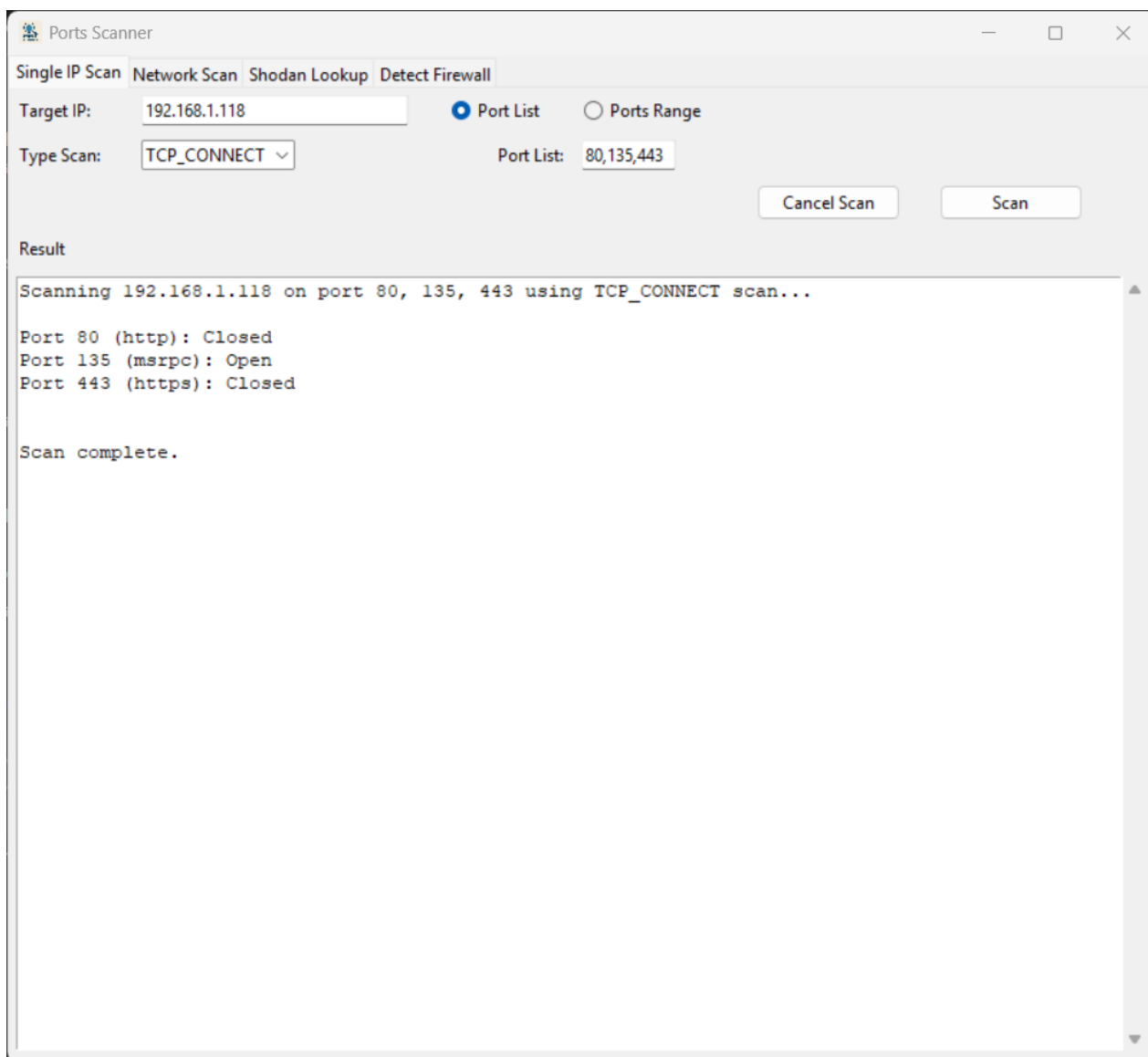
Chương này đã trình bày mô tả các hàm chức năng các yếu tố giúp xác định yêu cầu của các chức năng và cách thức hoạt động chi tiết của từng chức năng chính của PSC.

CHƯƠNG 3. THỬ NGHIỆM VÀ ĐÁNH GIÁ

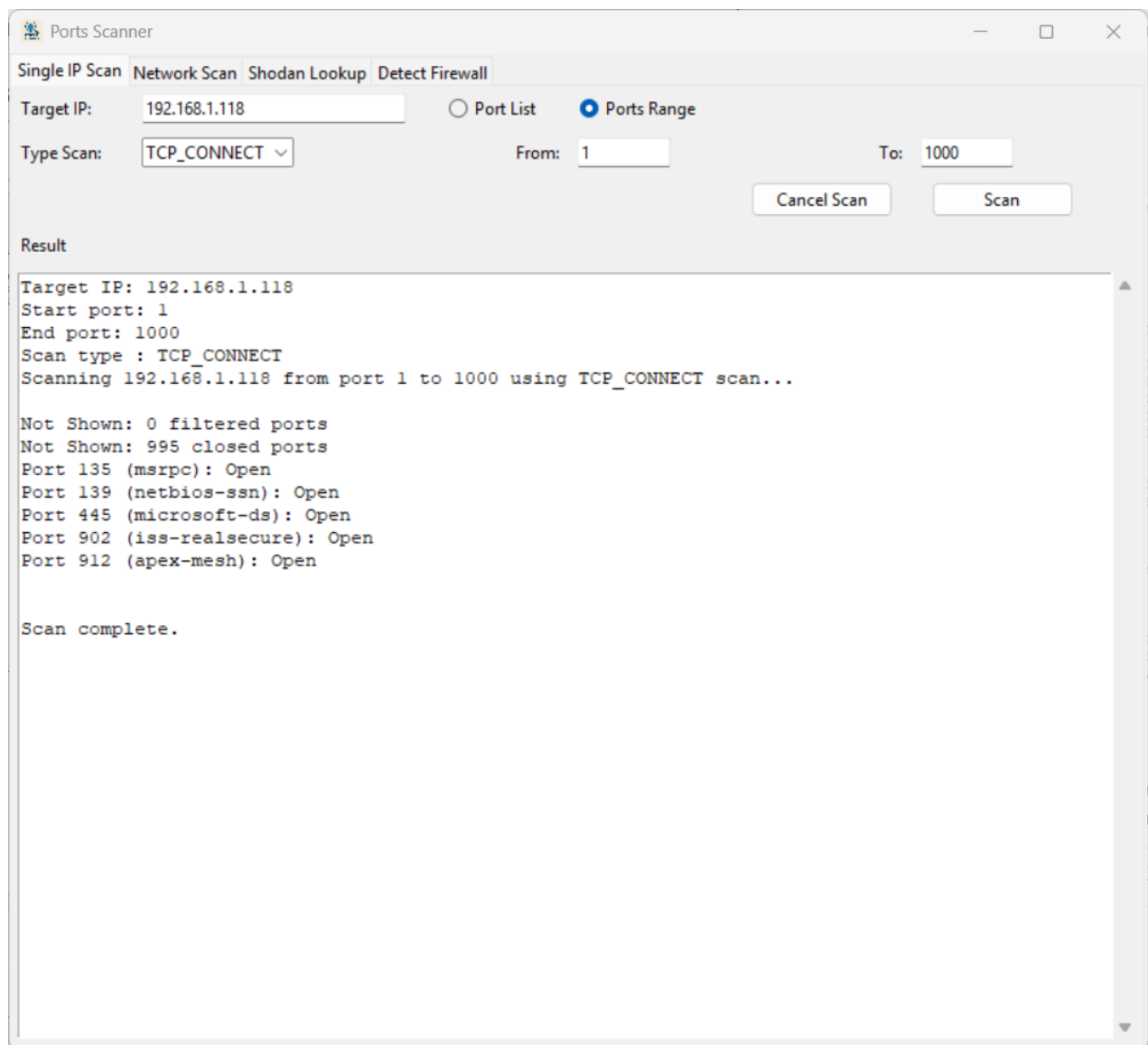
3.1 Triển khai thử nghiệm công cụ

3.1.1 Thử nghiệm với chức năng *Single IP Scan*

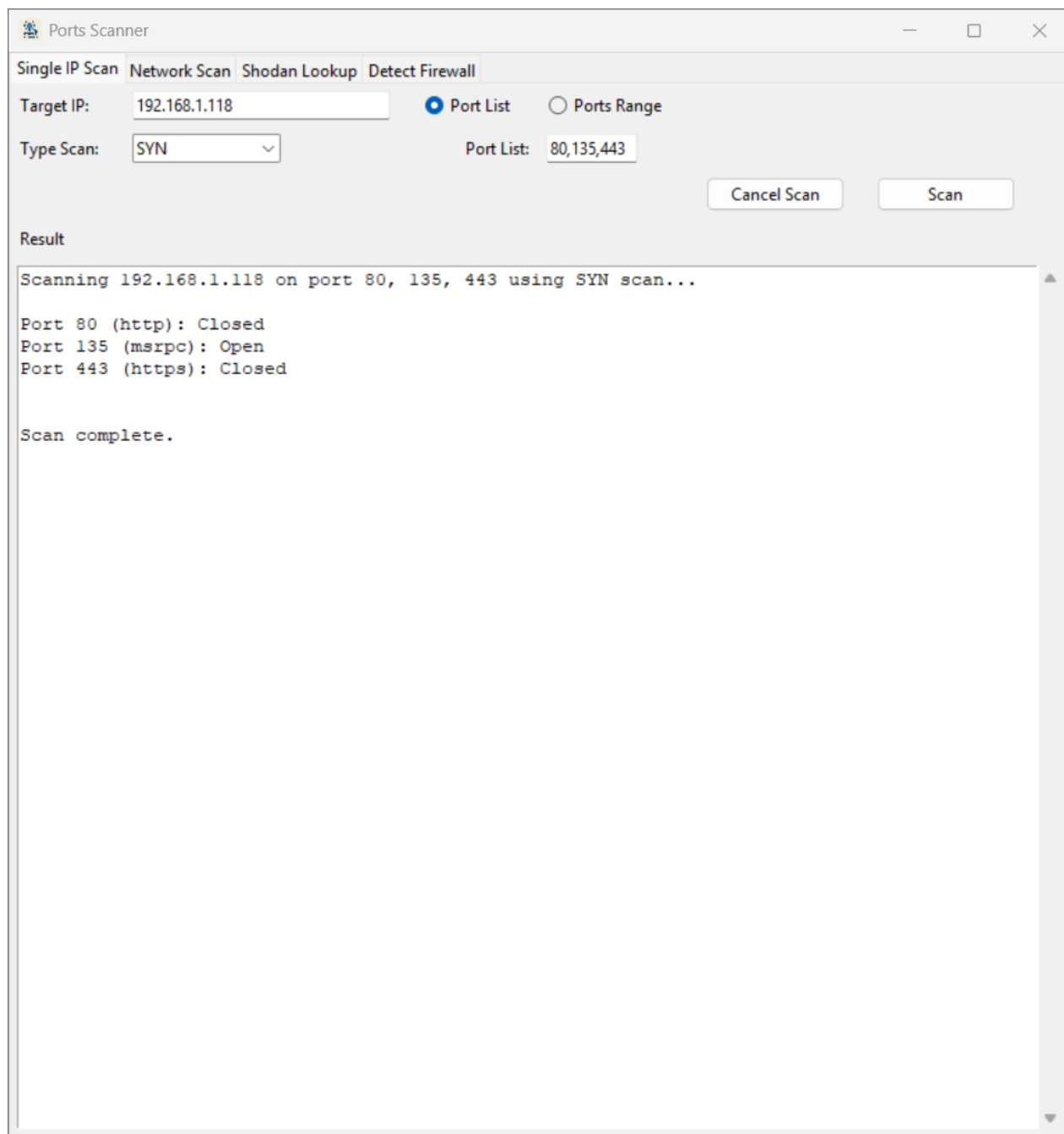
Chức năng Single IP Scan của PSC được triển khai trên thiết bị thực với địa chỉ IP 192.168.1.118. Hình 10 và 11 cung cấp giao diện của PSC của chức năng Single IP Scan thực hiện quét với địa chỉ 192.168.1.118 và 8.8.8.8 với các cổng 80,135,443 và dải cổng từ 1-1000 sử dụng loại quét là SYN Scan. Với những kiểu quét khác có thể cho ra những kết quả khác nhau.



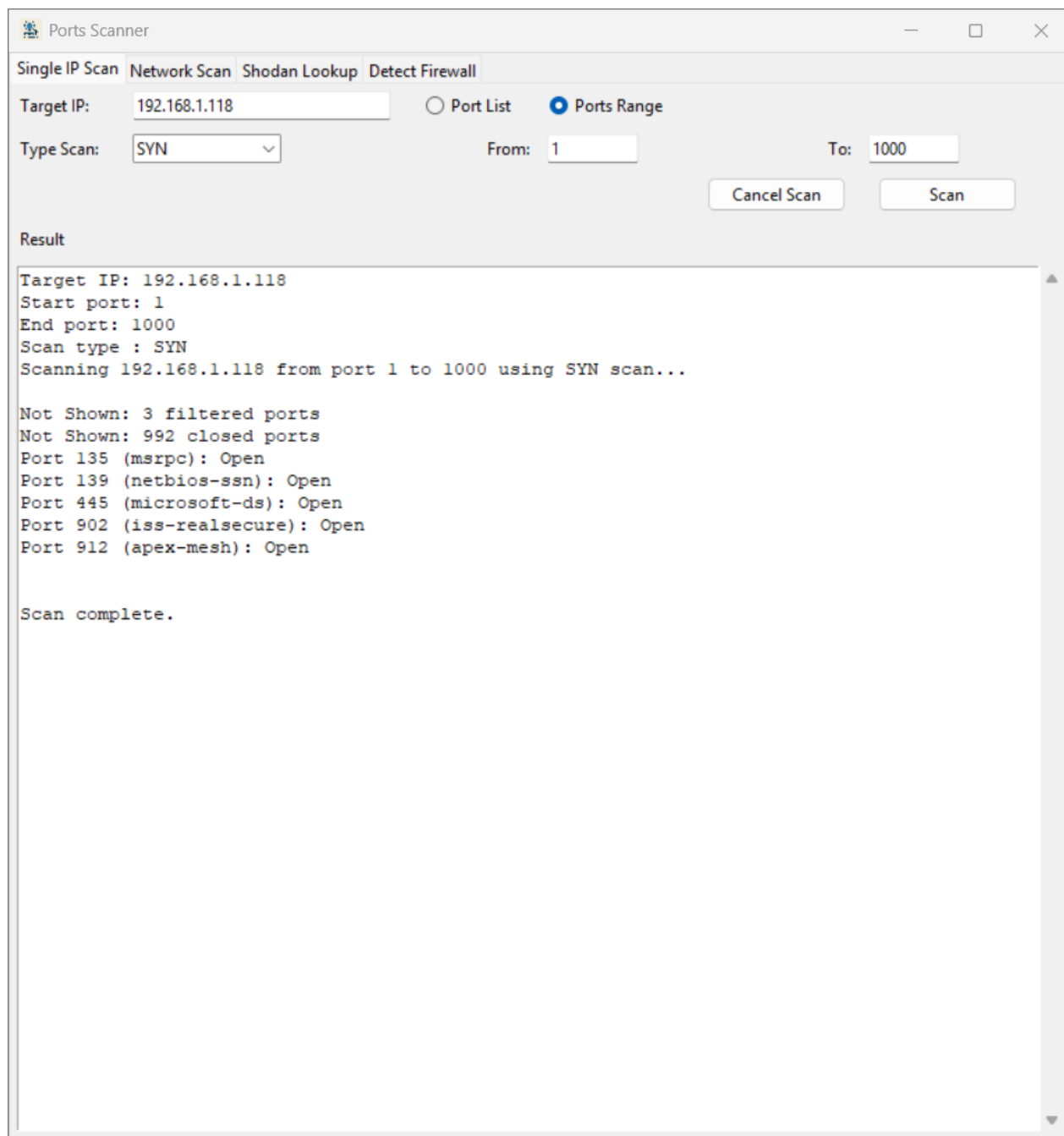
Hình 10: Giao diện Single IP Scan với 1 số cổng (TCP Connect Scan)



Hình 11: Giao diện Single IP Scan với 1 dải cổng (TCP Connect Scan)

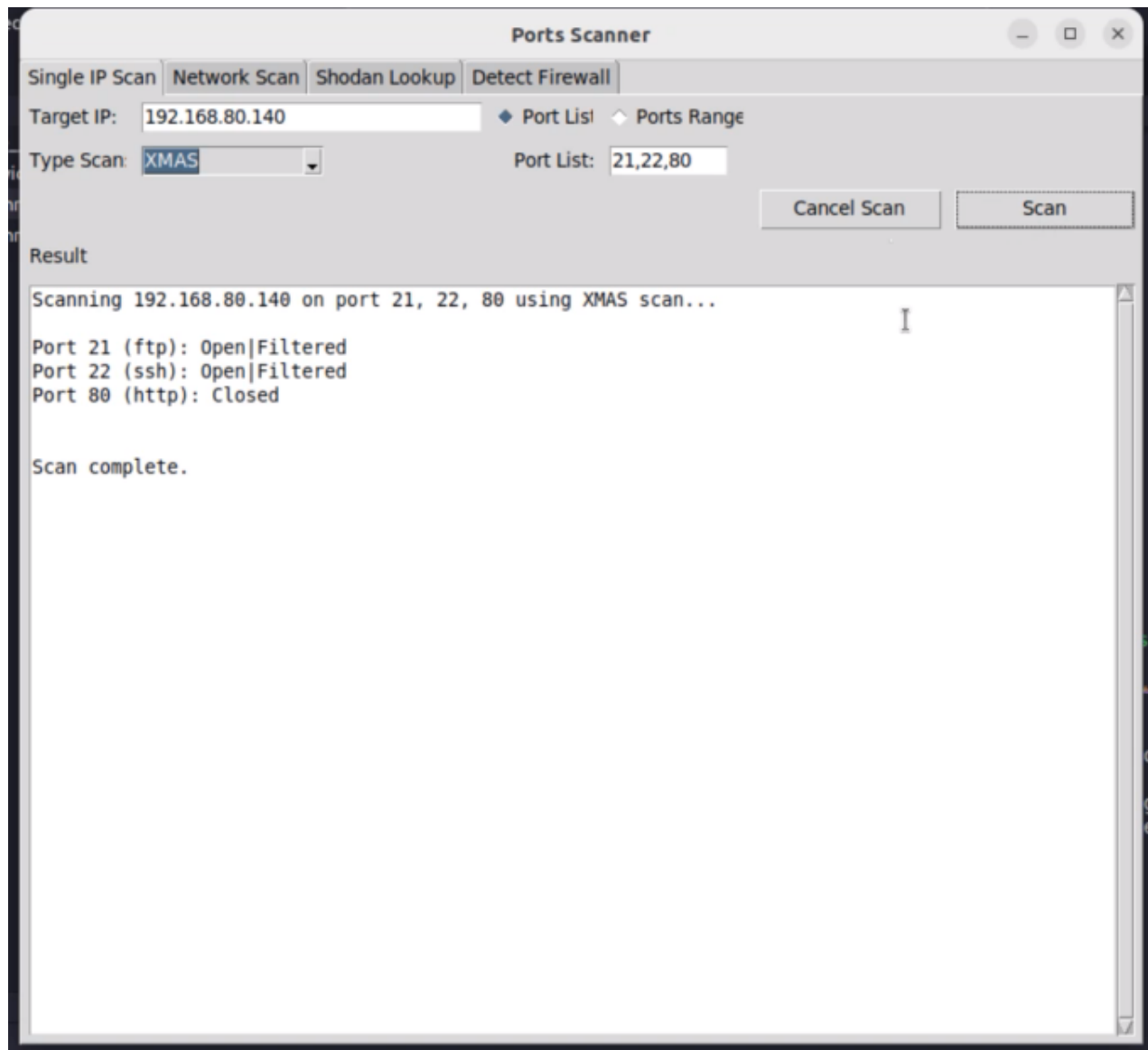


Hình 12: Giao diện Single IP Scan với 1 số cổng (SYN Scan)

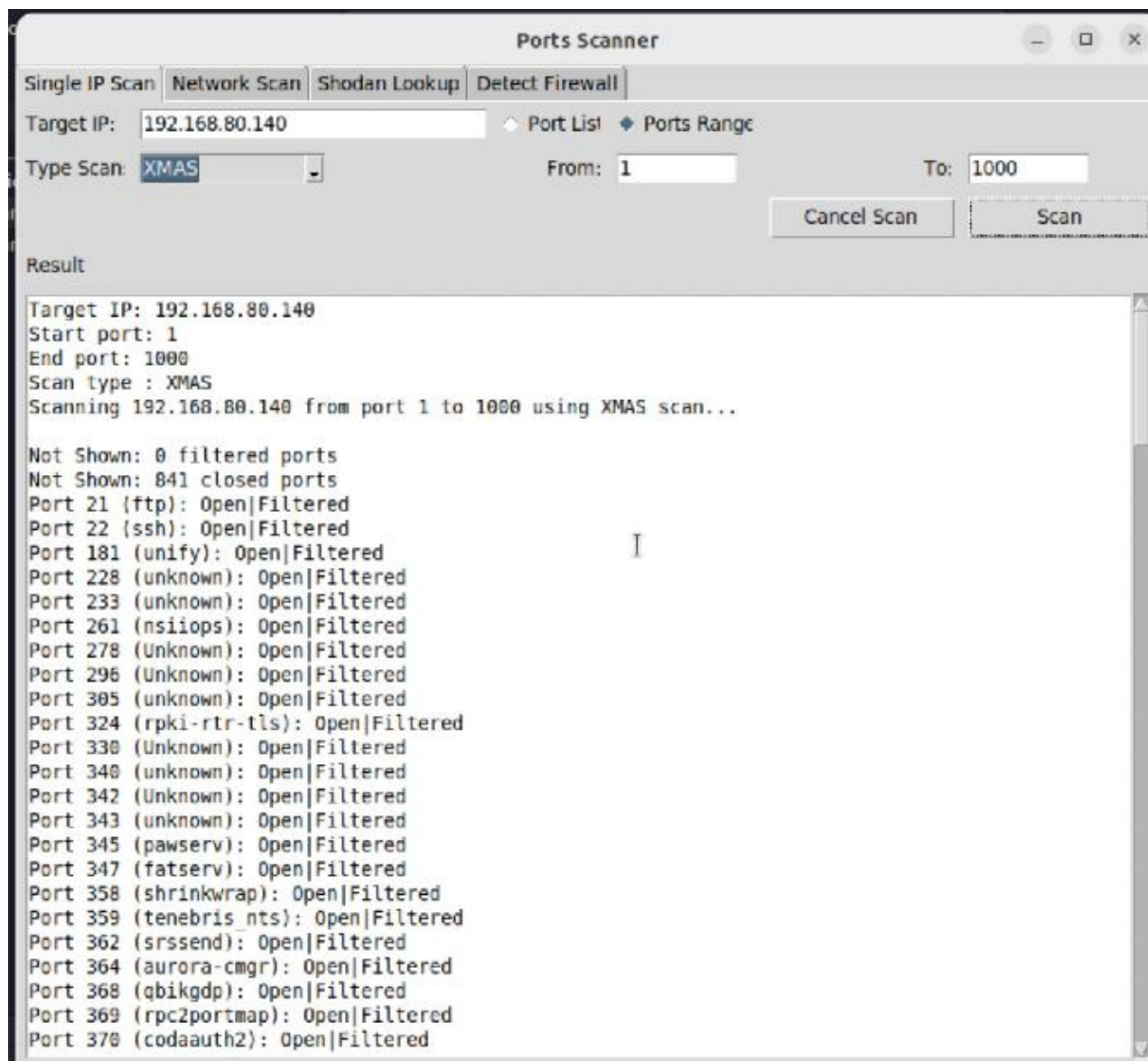


Hình 13: Giao diện Single IP Scan với 1 dải cổng (SYN Scan)

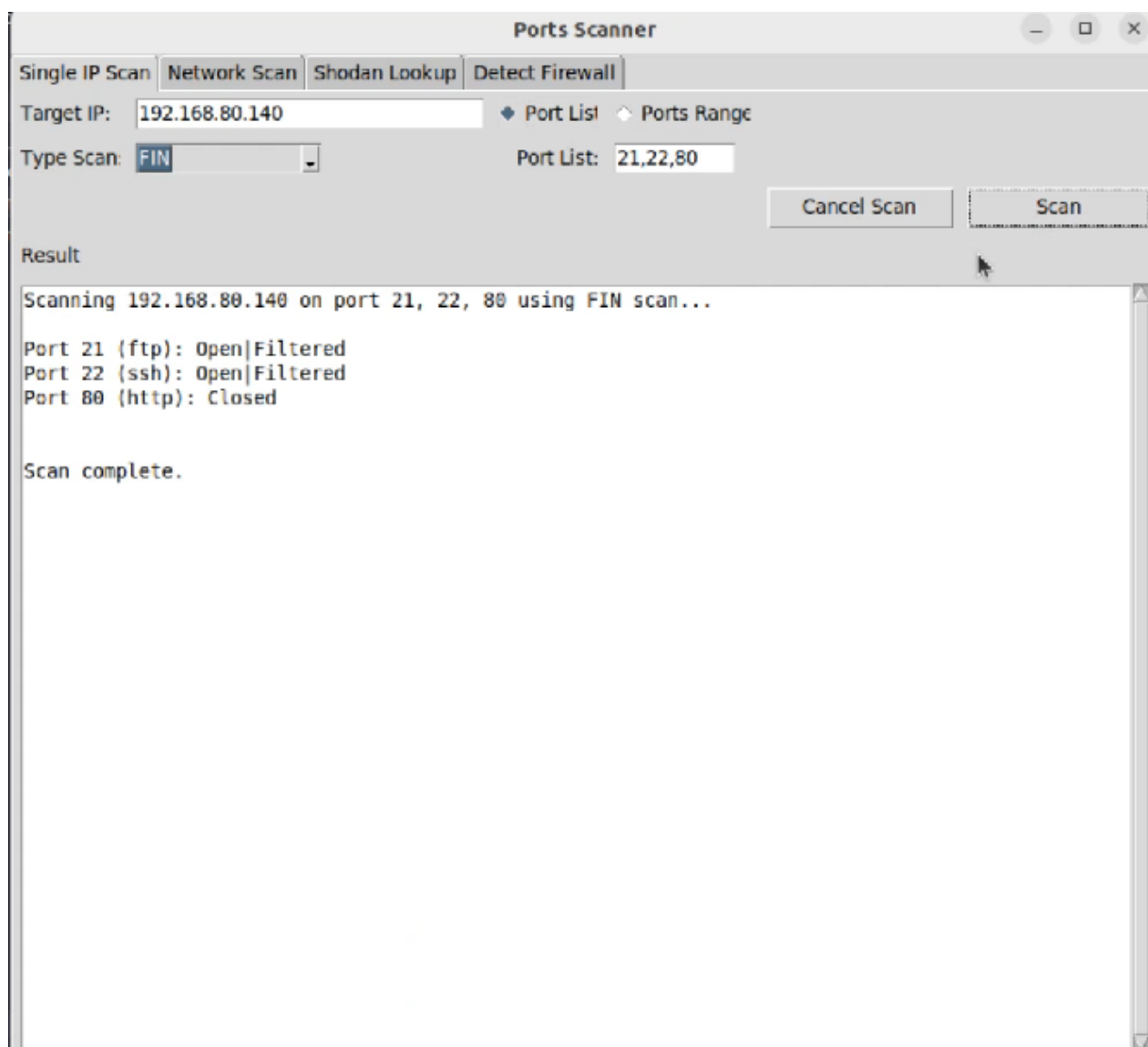
Do tính chất của XMAS, FIN, NULL, UDP Scan đã nêu ở phần mô tả, demo sẽ được thực hiện trong môi trường máy ảo Linux với máy đích bị quét là Kali Linux với địa chỉ IP 192.168.80.140.



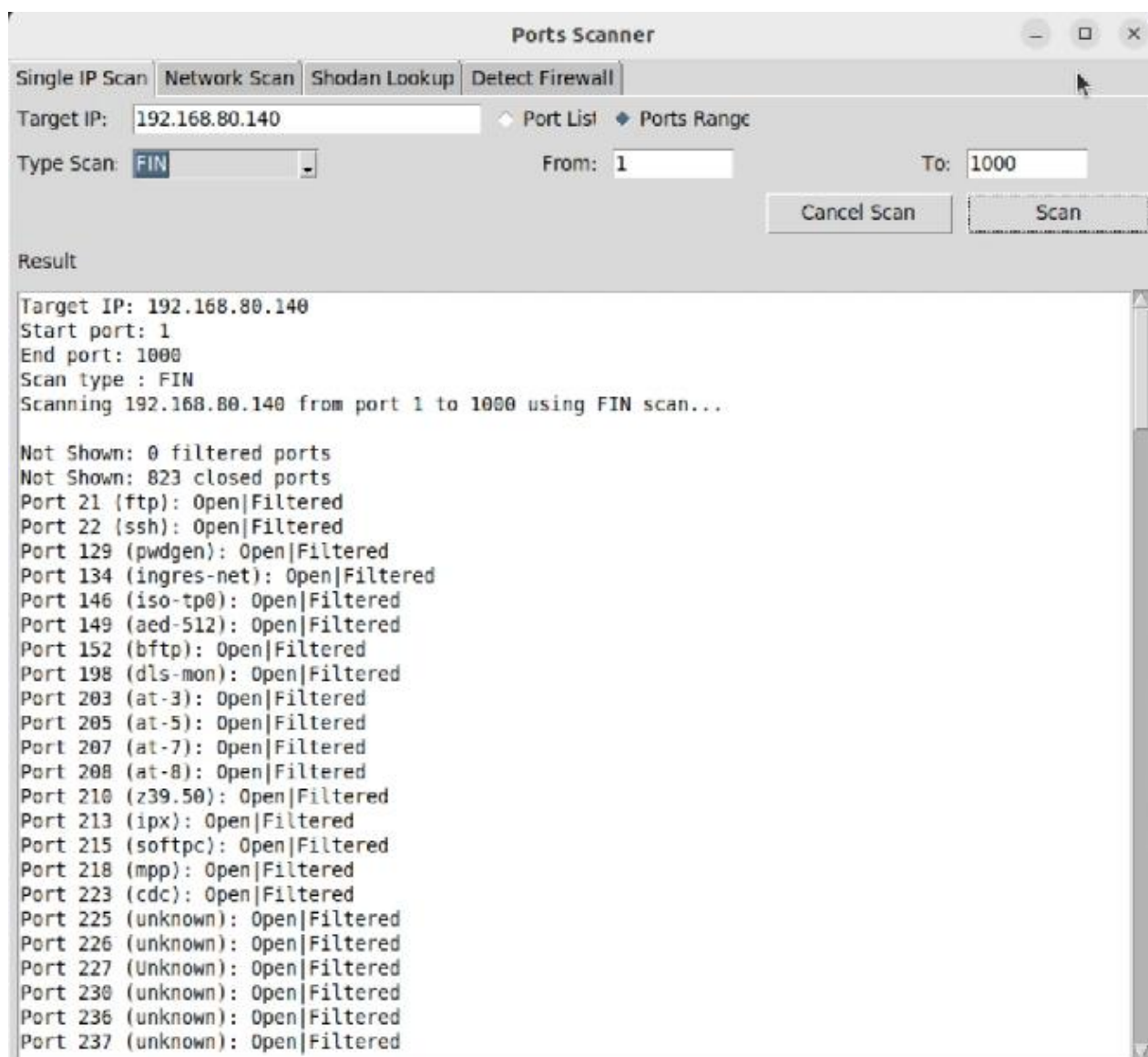
Hình 14: Giao diện Single IP Scan với 1 số cổng (XMAS Scan)



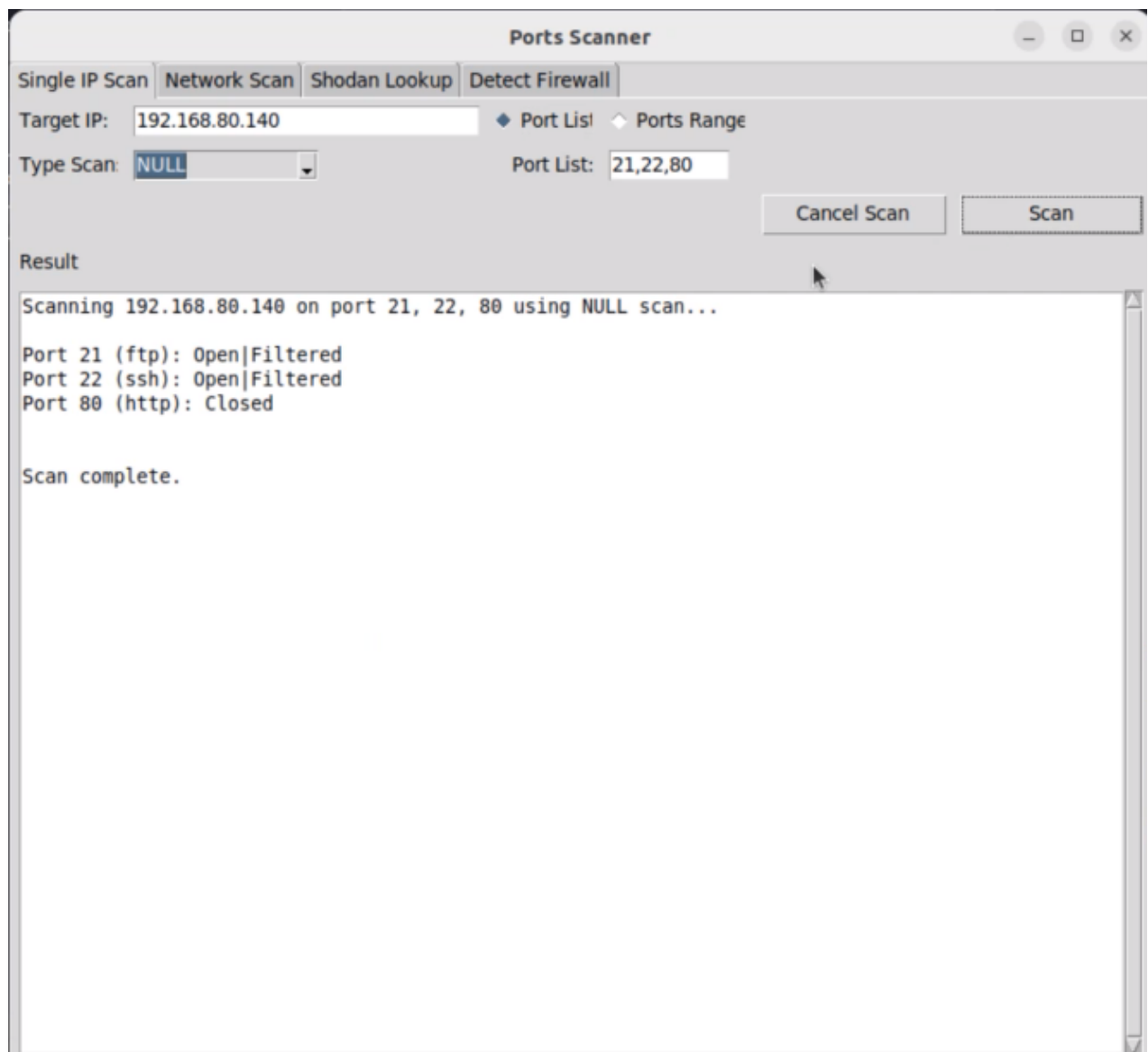
Hình 15: Giao diện Single IP Scan với 1 dải cổng (XMAS Scan)



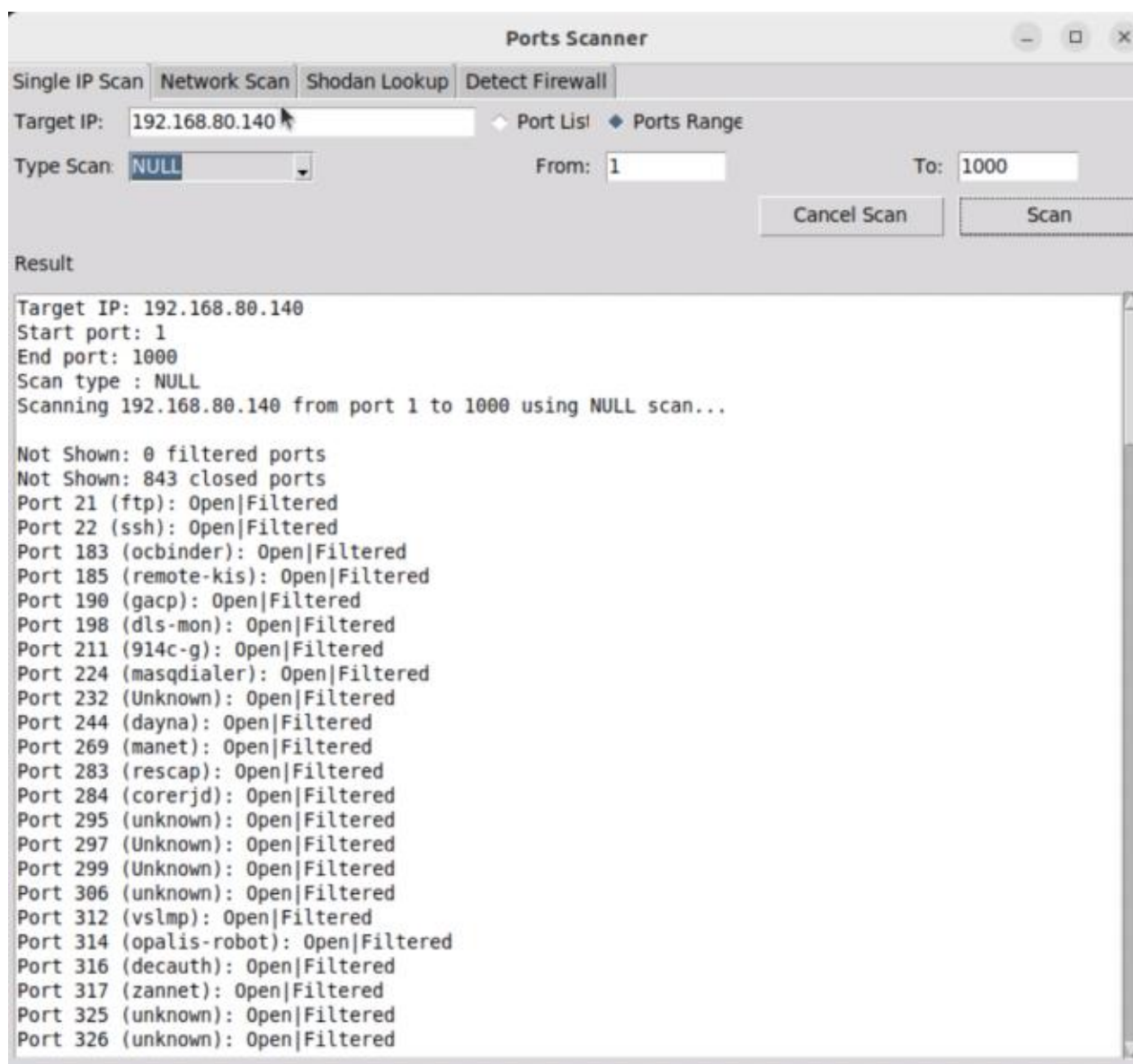
Hình 16: Giao diện Single IP Scan với 1 số cổng (FIN Scan)



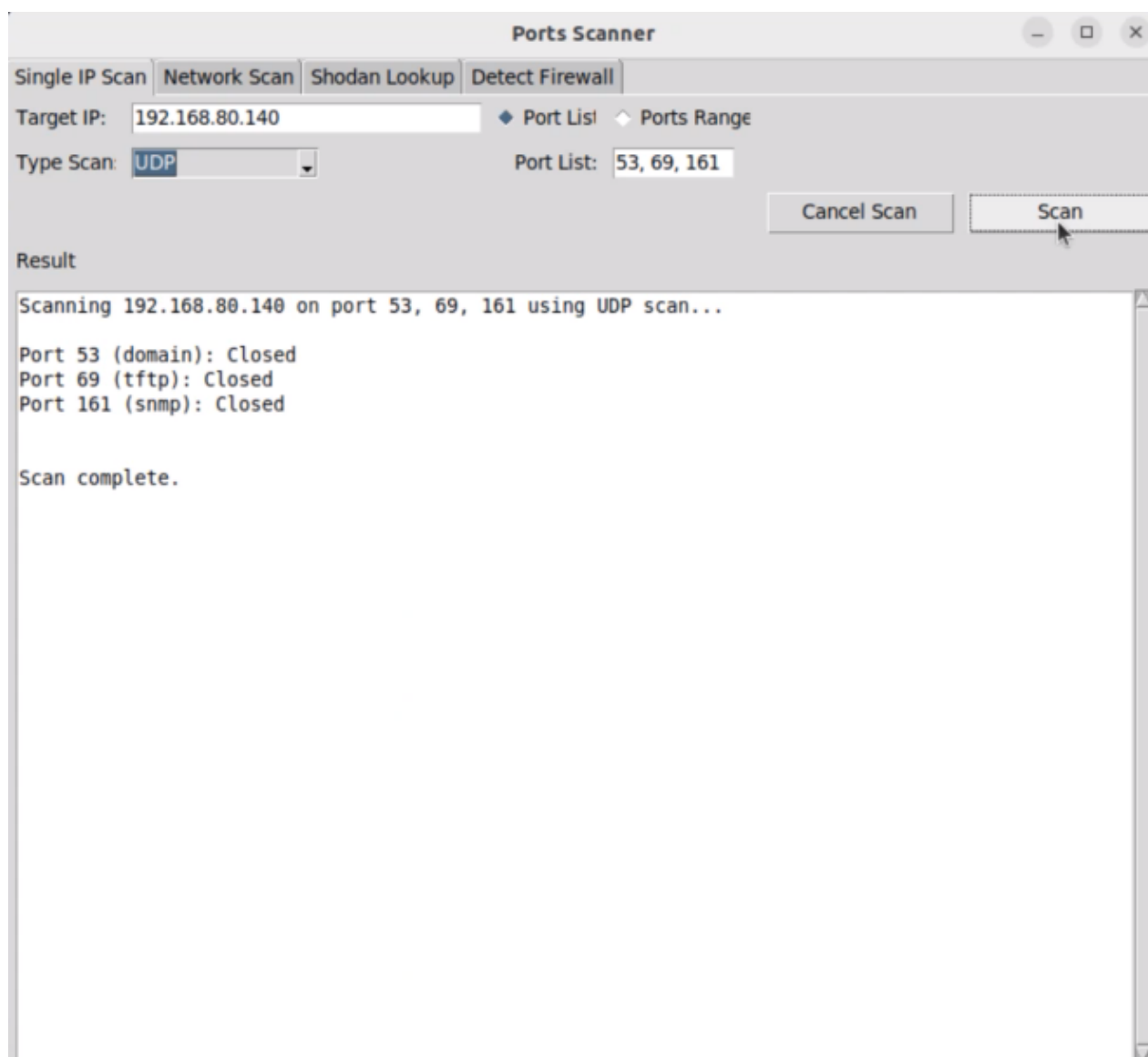
Hình 17: Giao diện Single IP Scan với 1 dải cổng (FIN Scan)



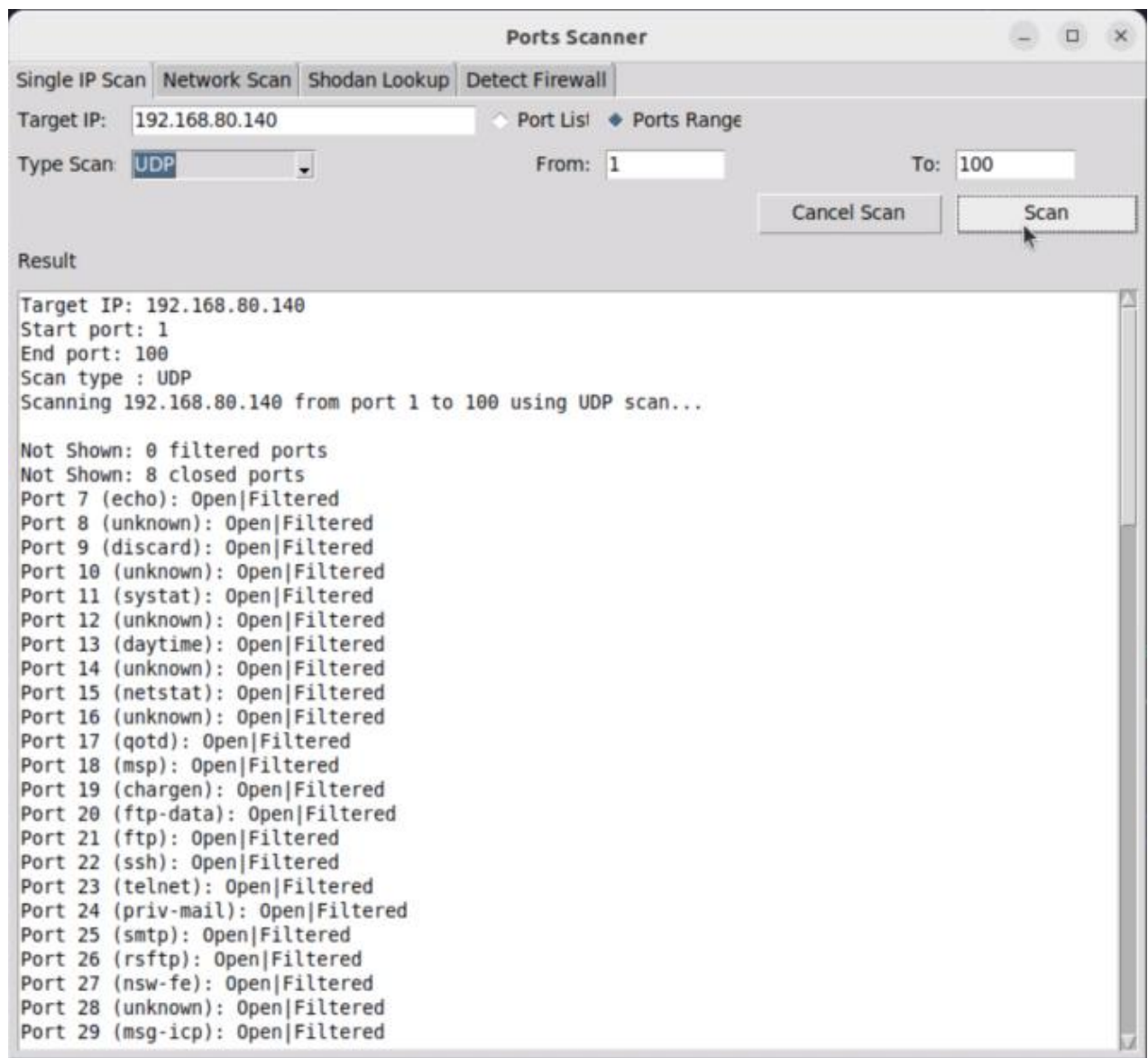
Hình 18: Giao diện Single IP Scan với 1 số cổng (NULL Scan)



Hình 19: Giao diện Single IP Scan với 1 dải cổng (NULL Scan)



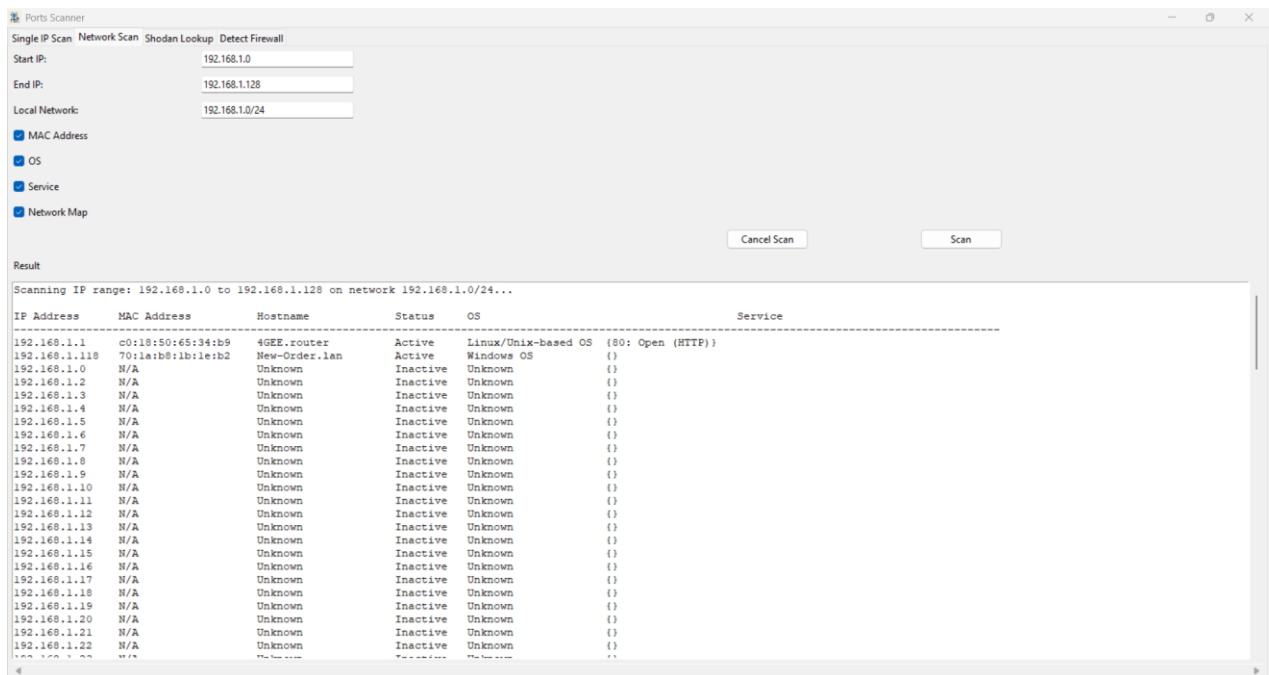
Hình 20: Giao diện Single IP Scan với 1 số cổng (UDP Scan)



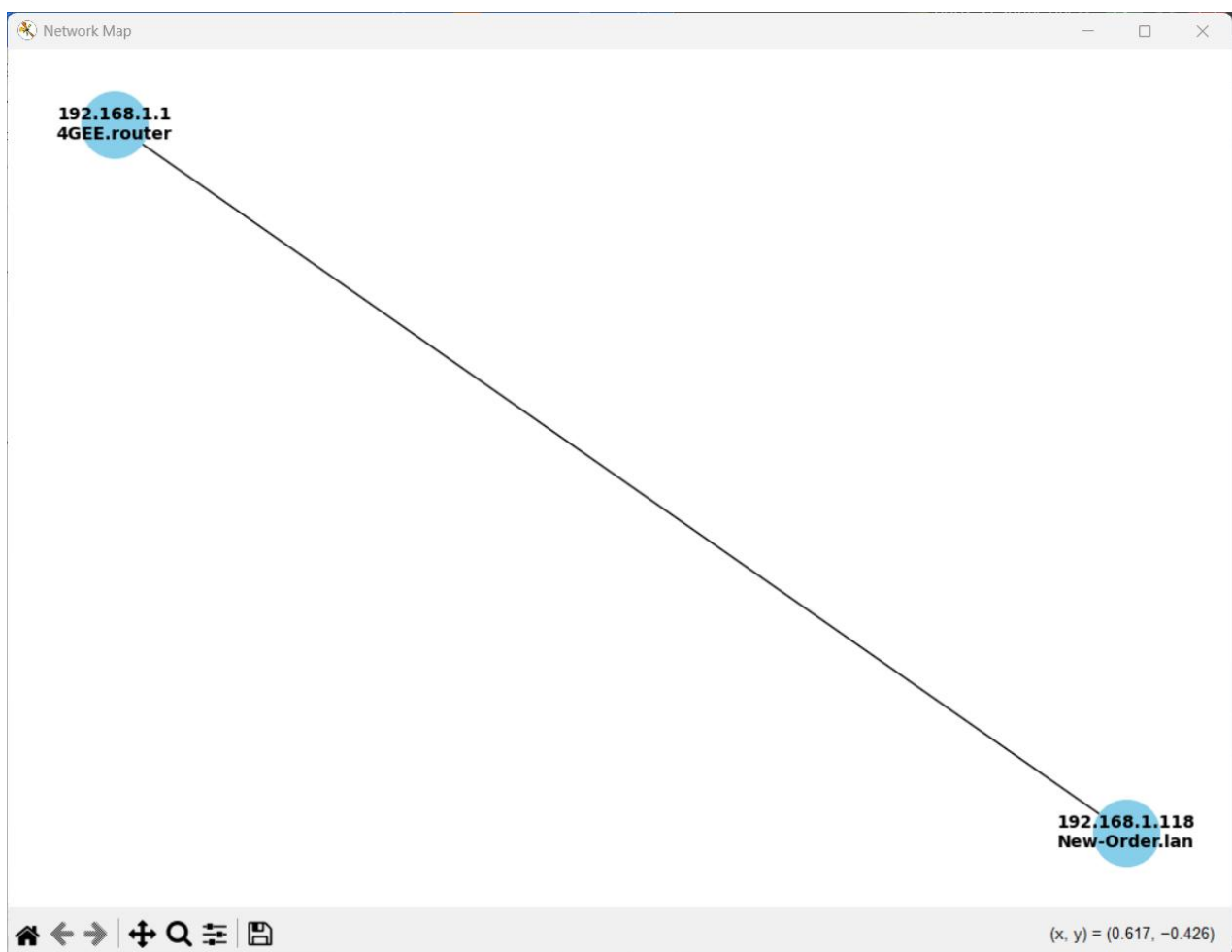
Hình 21: Giao diện Single IP Scan với 1 dải cổng

3.1.2 Thử nghiệm với chức năng Network Scan

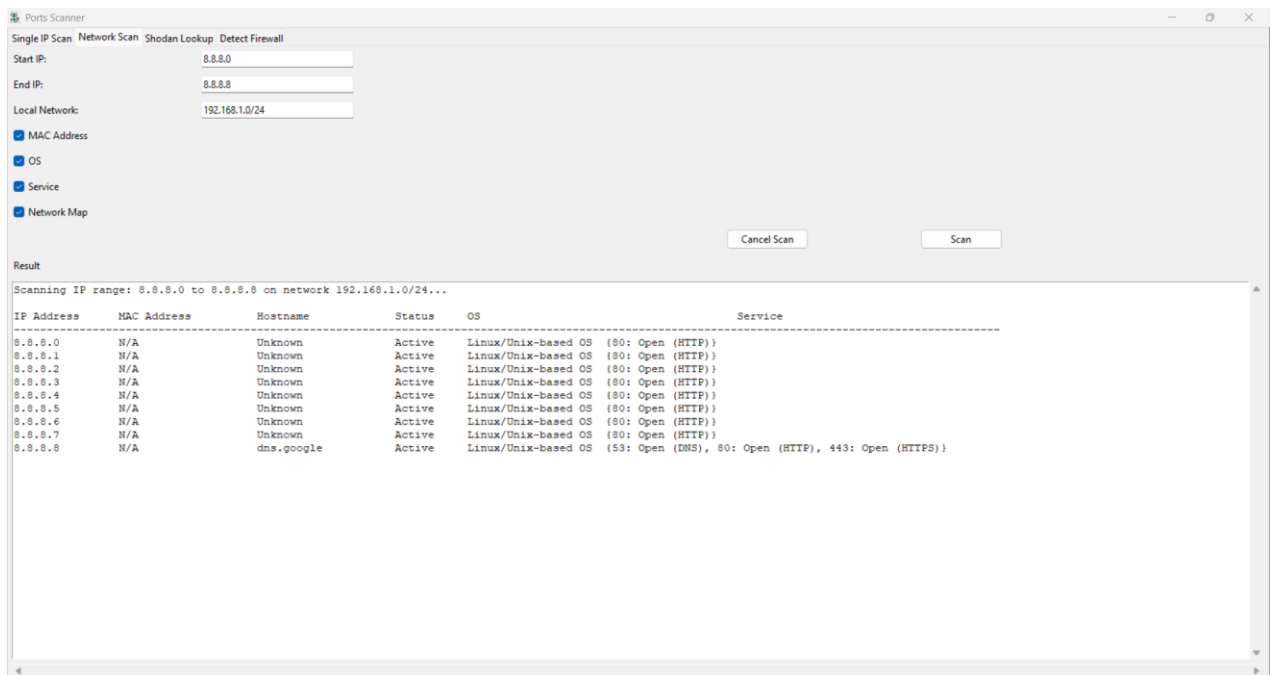
Chức năng Network Scan của PSC được triển khai trên thiết bị thực với địa chỉ IP 192.168.1.118. Hình 12, 13, 14 và 15 cung cấp giao diện của PSC của chức năng Network Scan thực hiện quét với dải địa chỉ IP từ 192.168.1.0 đến 192.168.1.128 và dải địa chỉ 8.8.8.0 đến 8.8.8.8 với dải mạng cục bộ là 192.168.1.0/24.



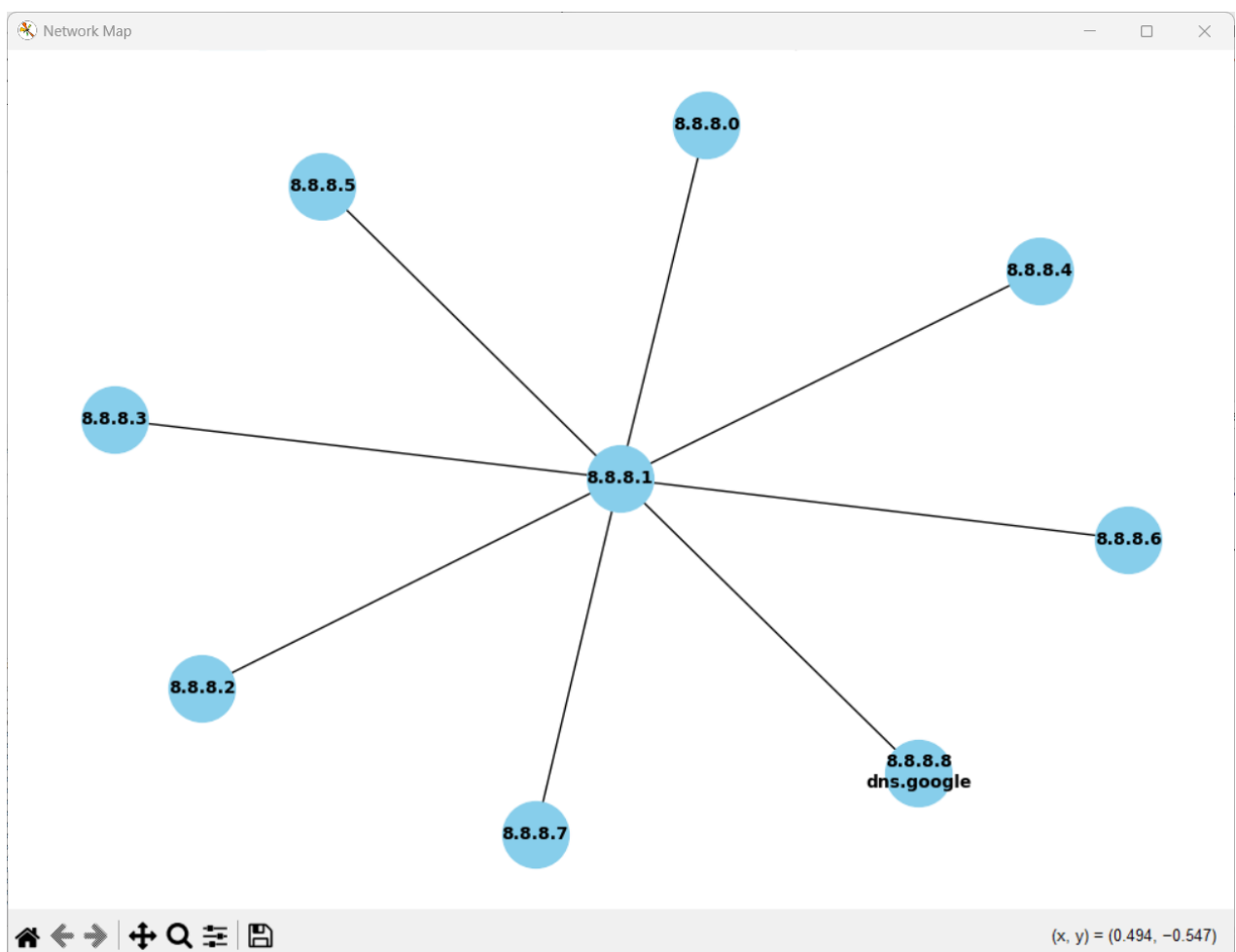
Hình 22: Giao diện Network Scan với dải địa chỉ trong mạng cục bộ



Hình 23: Đồ thị mạng với dải địa chỉ trong mạng cục bộ



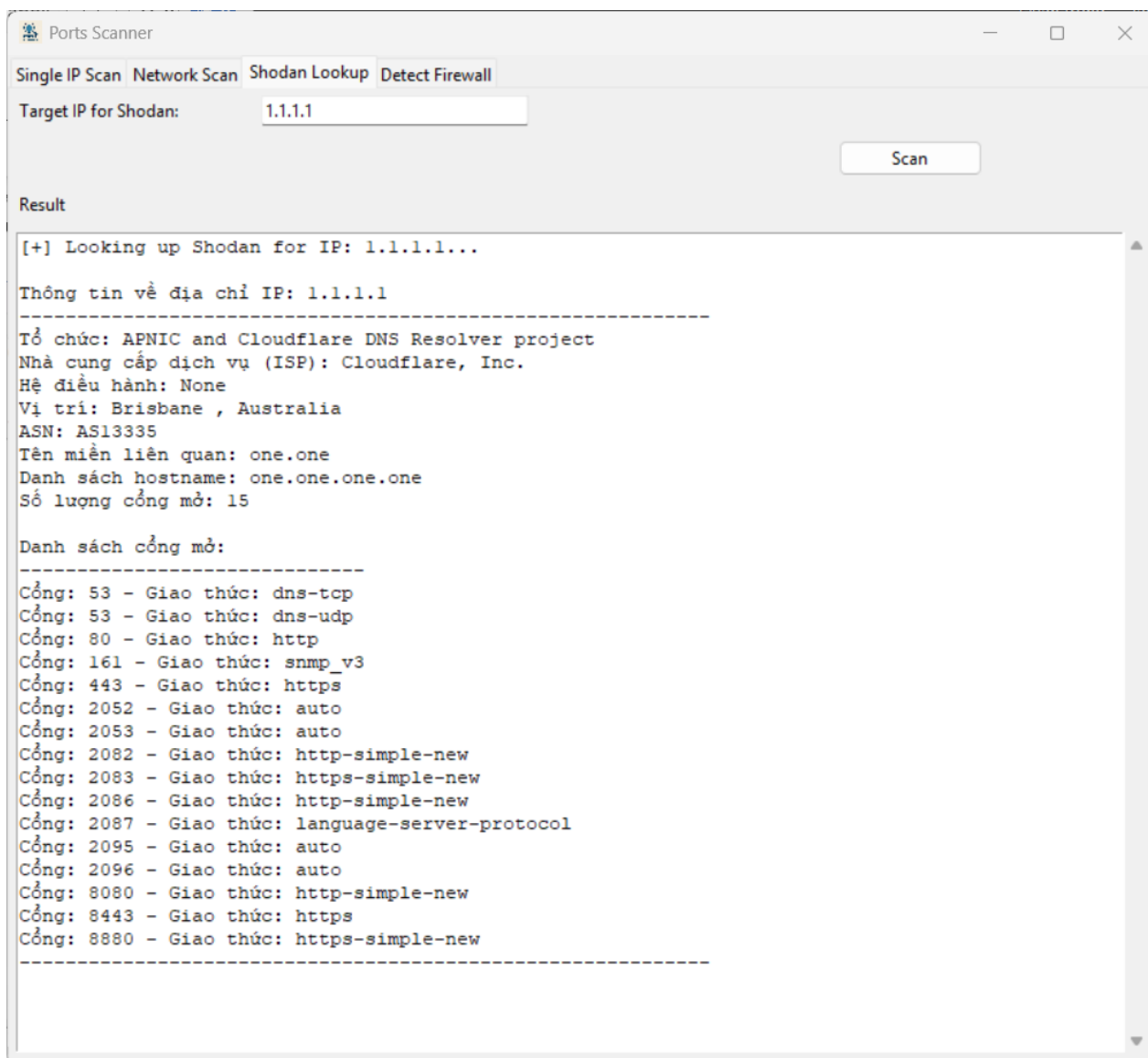
Hình 24: Giao diện Network Scan với dải địa chỉ ngoài mạng cục bộ



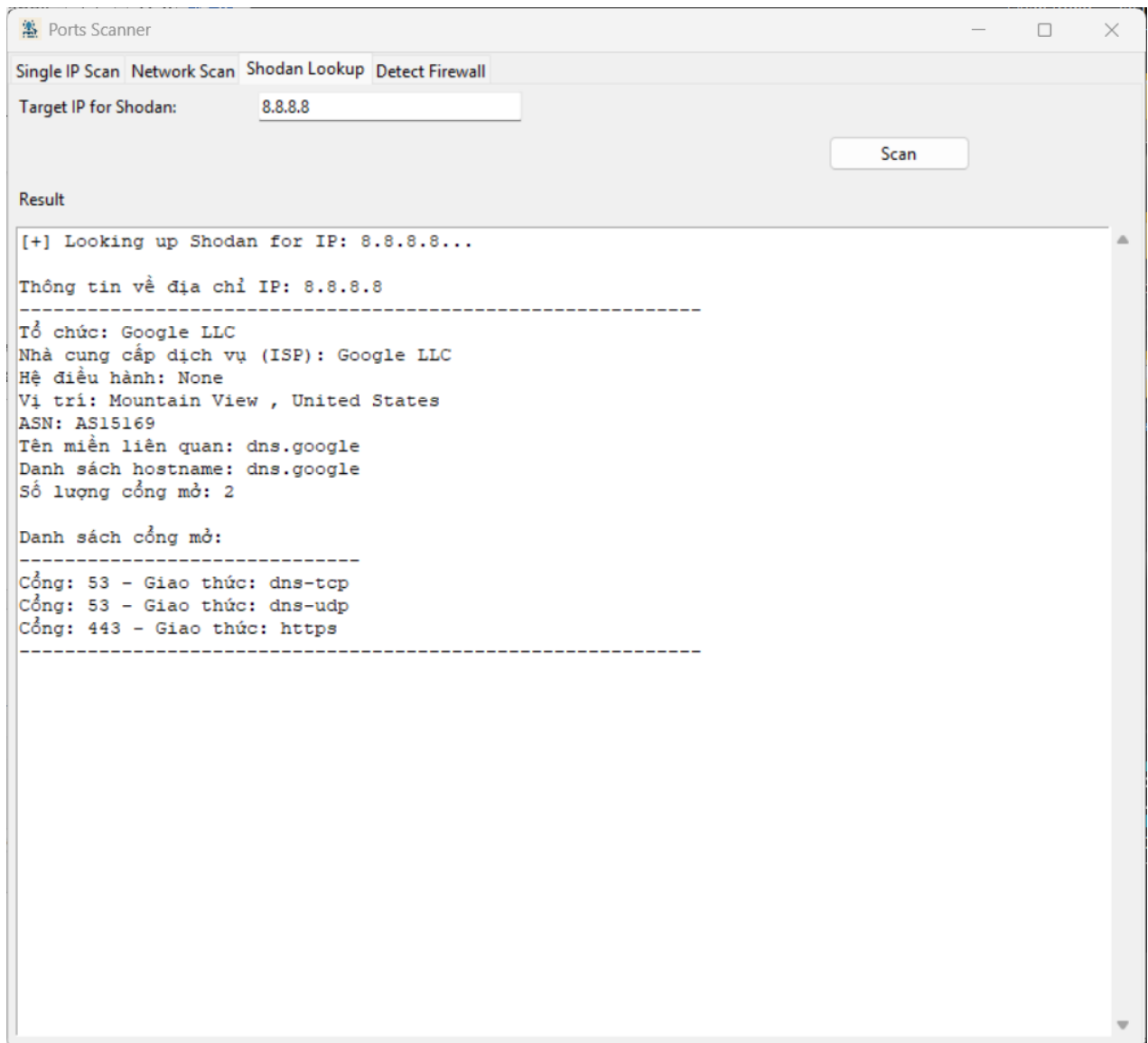
Hình 25: Sơ đồ mạng với dải địa chỉ ngoài mạng cục bộ

3.1.3 Thử nghiệm với chức năng Shodan Lookup

Chức năng Shodan Lookup Scan của PSC được triển khai trên thiết bị thực với địa chỉ IP 192.168.1.118. Hình 16 và 17 cung cấp giao diện của PSC của chức năng Shodan Lookup thực hiện quét với địa chỉ IP Public 1.1.1.1 và 8.8.8.8.



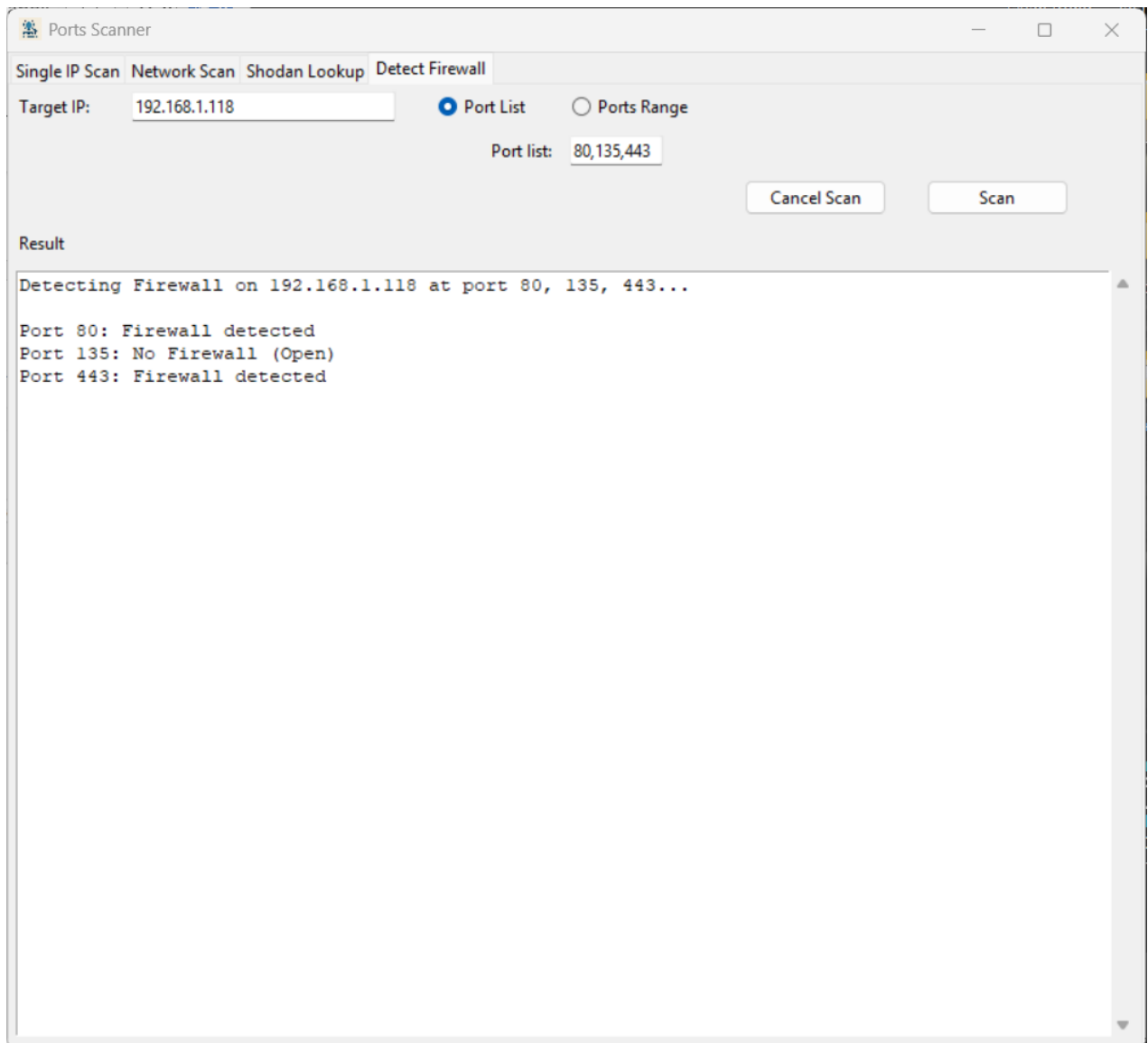
Hình 26: Giao diện Shodan Lookup với địa chỉ IP 1.1.1.1



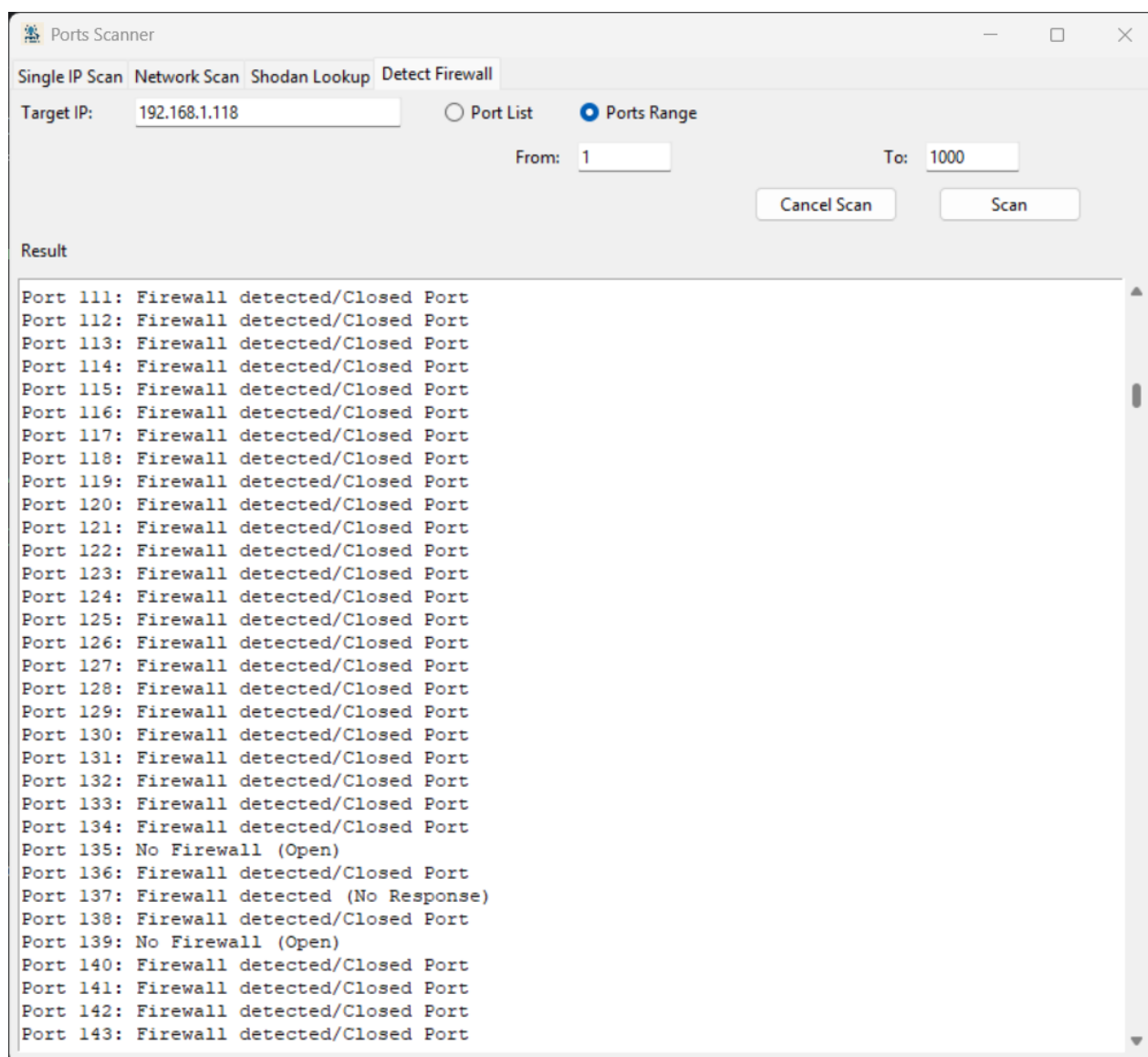
Hình 27: Giao diện Shodan Lookup với địa chỉ IP 8.8.8.8

3.1.4 Thử nghiệm với chức năng *Detect Firewall*

Chức năng Detect Firewall Scan của PSC được triển khai trên thiết bị thực với địa chỉ IP 192.168.1.118. Hình 18 và 19 cung cấp giao diện của PSC của chức năng Detect Firewall thực hiện quét với địa chỉ 192.168.1.118 với các cổng 80,135,443 và dải cổng từ 1-1000.



Hình 28: Giao diện Detect Firewall với 1 số cổng



Hình 29: Giao diện Detect Firewall với 1 dải cổng

3.2 Kết chương

Chương này tiến hành thực nghiệm từng chức năng với những địa chỉ IP cụ thể. Những chức năng của PSC giúp cho người dùng xác định cổng mở, các thiết bị hoạt động cùng với những thông tin kèm theo, tra cứu thông tin của địa chỉ IP Public và phát hiện tường lửa thiết bị ứng với những cổng cần tìm.

KẾT LUẬN

Các kết quả đạt được

Nhóm thực hiện đề tài “Xây dựng công cụ quét cổng Ports Scanner” đã hoàn thành việc xây dựng, tùy biến và thử nghiệm thành công 4 chức năng chính của công cụ. Đề tài đã thực hiện đầy đủ các nội dung đã đăng kí theo đề cương như sau:

- Xây dựng các chức năng chính của công cụ PSC.
- Thiết kế và cài đặt các chức năng chính của PSC bao gồm: Single IP Scan, Network Scan, Shodan Lookup, Detect Firewall.
- Thử nghiệm và đánh giá các chức năng chính của PSC bao gồm: Single IP Scan, Network Scan, Shodan Lookup, Detect Firewall.

Hướng phát triển

Đề tài này có thể được mở rộng theo các hướng sau:

- Phát triển các hàm mở rộng nhằm thu thập thêm và chính xác hơn các thông tin từ những thiết bị mục tiêu và quét ẩn nhằm tránh bị phát hiện.
- Phát triển thêm chức năng xuất các thông tin tìm được ra thành file dữ liệu theo yêu cầu của người dùng.
- Cải thiện tốc độ quét cổng của các phương thức.
- Phát triển thêm nhiều hình thức quét nâng cao.

TÀI LIỆU THAM KHẢO

- [1] Ths. Nguyễn Hoàng Anh, Ths. Đặng Ngọc Hùng, Bài giảng lập trình với Python, năm 2022.
- [2] Phạm Hoàng Duy, Giáo trình An toàn hệ điều hành, năm 2017.
- [3] <https://viblo.asia/p/nmap-port-scan-cac-phuong-phap-quet-cong-tu-co-ban-den-nang-cao-gDVK2PMwILj>, truy cập vào lúc 15:31 ngày 5 tháng 3 năm 2025.
- [4] <https://stackoverflow.com>, truy cập vào lúc 21:46 ngày 11 tháng 4 năm 2025.
- [5] <https://aws.amazon.com/vi/what-is/icmp/>, truy cập vào lúc 20:57 ngày 12 tháng 4 năm 2025.
- [6] <https://ipccisco.com>, truy cập vào lúc 21:58 ngày 12 tháng 4 năm 2025.
- [7] <https://www.techtarget.com>, truy cập vào lúc 22:33 ngày 12 tháng 4 năm 2025.
- [8] <https://www.shodan.io>, truy cập vào lúc 23:01 ngày 13 tháng 4 năm 2025.
- [9] <https://nmap.org>, truy cập vào lúc 18:50 ngày 14 tháng 4 năm 2025.